

“瑕宝阁”微瑕非遗产品交易平台

——商品模块测试设计与执行报告

团队编号：第 9 组

项目成员：2251310 杜天乐 2251916 罗诗雨 2252443 吴婉宁 2252444 赵思源

1. 核心功能（商品模块）测试方案与用例设计

本测试文档以应用的核心——“商品”模块为中心，旨在系统性地验证其完整功能。为确保对商品从上架、被用户发现、直至最终完成购买的整个生命周期进行全面测试，本次测试范围将完整覆盖从**用户认证**开始，到**商品交互**，再到**交易支付**结束的整个业务流程。本章节将详细阐述测试的总体方案、覆盖的测试用例概览，以及所采用的测试技术。

1.1 测试目标与范围

1.1.1 测试目标

本次测试的核心目标是全面验证**商品模块**及其完整业务流程的功能正确性、健壮性与代码质量，确保为用户提供无缝、可靠的购物体验。

1.1.2 测试范围

本次测试的范围构成了一个完整的业务闭环：

- 流程起点 - 用户认证**：用户登录，获取进行后续操作的身份凭证。
- 流程核心 - 商品交互**：
 - 商品浏览与分类筛选
 - 商品详情展示
 - 导航栏商品/商户搜索
 - 卖家新增商品
- 流程终点 - 交易与支付**：
 - 创建订单
 - 使用钱包或支付宝完成支付
 - 买家进行钱包充值

1.2 测试用例设计总览

为全面测试上述功能，我们综合运用了多种黑盒与白盒测试技术。下表概括了本次设计的主要测试用例，详细的实现与执行分析见后续章节。

测试阶段	核心功能/流程	测试方法	关键测试点
黑盒测试	登录接口	等价类、边界值	用户名/密码的有效、无效、边界、空值情况

测试阶段	核心功能/流程	测试方法	关键测试点
黑盒测试	导航栏搜索	等价类、边界值	关键词与搜索类型的有效、无效、边界组合
黑盒测试	新增商品	决策表	覆盖商品各字段（价格、分类、图片等）所有有效/无效的逻辑组合
黑盒测试	确认订单与支付	等价类、边界值	订单号、支付金额、收货信息的有效、无效、边界值
黑盒测试	钱包充值	等价类、边界值	用户ID、充值金额、回调URL的有效、无效、边界值
白盒测试	商品浏览与搜索(前端)	组件测试、分支覆盖	异步数据加载、分页、空/异常状态、搜索结果展示
白盒测试	商品详情(前端)	状态迁移、分支覆盖	价格展示逻辑、图片切换、收藏/取消收藏状态变更
白盒测试	购买与支付(前端)	状态迁移、分支覆盖	地址修改、积分使用、钱包/支付宝支付流程、异常处理（余额不足）
白盒测试	下单/搜索(后端)	路径/分支覆盖	商品存在/已售/折扣等下单分支；商品/店铺搜索结果分支
白盒测试	商品增改(后端)	路径/分支/边界覆盖	新增/编辑商品时对分类、标签等关联数据的复杂更新逻辑及边界情况

1.3 测试覆盖率综合分析

本测试方案旨在从多个维度实现高水平的测试覆盖，以确保软件的整体质量。

- **流程覆盖 (Flow Coverage):** 用例设计完整地贯穿了用户的核心使用路径（登录 -> 商品浏览 -> 支付），并对钱包充值、新增商品等关键旁路流程进行了端到端验证，确保了关键业务场景的完整性。
- **数据与条件覆盖 (Data & Condition Coverage):**
 - 在**黑盒测试**中，通过等价类、边界值和决策表，系统性地覆盖了所有API输入参数的有效数据、典型无效数据和关键边界条件。
 - 在**白盒测试**中，通过精心设计的用例，旨在实现高水平的**语句覆盖 (Statement Coverage)** 和**分支覆盖 (Branch Coverage)**，确保代码中的绝大部分可执行语句和逻辑判断（如 `if/else`）都至少被执行一次。
- **异常路径覆盖 (Exception Path Coverage):** 我们主动设计了大量的负面用例，旨在触发系统预期的业务错误（如用户不存在、余额不足）和技术异常（如非法参数、数据库访问失败）。这旨在验证系统能否在各种异常情况下保持稳定，并提供清晰、统一的错误反馈，而非直接崩溃。

2. 黑盒测试

2.1 黑盒测试 - 前端Selenium

2.1.1 测试概述

- **测试工具**: Python + Selenium + unittest + pytest-html
- **浏览器驱动**: ChromeDriver
- **测试页面**: 本地部署页面 `http://localhost:5173`
- **测试目标**: 验证用户从登录到支付整个购买流程的正确性与可用性

2.2.2 测试环境

项目	配置
操作系统	Windows
浏览器	Chrome
Python 版本	3.x
WebDriver	ChromeDriver
第三方库	selenium, pytest, pytest-html, unittest

2.2.3 测试流程与结果

测试步骤	操作描述	检查点 / 断言说明	结果
登录	输入账号 <code>buyer1@gg.com</code>	登录成功跳转首页，控制台打印信息	成功
钱包充值	输入金额 1000，跳转支付页	成功跳转至支付宝模拟页面	成功
支付操作	输入支付信息并提交	成功返回主窗口	成功
商品搜索	搜索“银”，点击商品	商品详情页成功加载	成功
下单购买	填写收货信息，点击支付	页面跳转到支付方式选择界面	成功

2.2.4 测试日志摘要

2025-06-08 14:57:42,986 - INFO - === 测试组初始化 ===
2025-06-08 14:57:44,794 - INFO -
=== 开始测试: test_login ===
2025-06-08 14:57:45,291 - INFO - 访问登录页面
2025-06-08 14:57:47,309 - INFO - 输入邮箱
2025-06-08 14:57:48,410 - INFO - 输入密码
2025-06-08 14:57:49,484 - INFO - 点击登录按钮
2025-06-08 14:57:51,517 - INFO - 登录成功
2025-06-08 14:57:51,517 - INFO - 登录测试通过
2025-06-08 14:57:51,517 - INFO - 测试耗时: 0:00:06.723473
2025-06-08 14:57:51,517 - INFO - === 结束测试: test_login ===

2025-06-08 14:57:51,520 - INFO -
=== 开始测试: test_product_purchase ===
2025-06-08 14:57:51,752 - INFO - 访问登录页面
2025-06-08 14:57:53,764 - INFO - 输入邮箱
2025-06-08 14:57:54,859 - INFO - 输入密码
2025-06-08 14:57:56,045 - INFO - 点击登录按钮
2025-06-08 14:57:58,082 - INFO - 登录成功
2025-06-08 14:57:58,084 - INFO - 开始测试: 商品搜索
2025-06-08 14:57:58,569 - INFO - 访问商品页面
2025-06-08 14:58:00,596 - INFO - 输入搜索关键词: 瓷
2025-06-08 14:58:00,786 - INFO - 点击搜索按钮
2025-06-08 14:58:02,947 - INFO - 点击商品: 大号
2025-06-08 14:58:25,549 - INFO - 测试耗时: 0:00:34.029436
2025-06-08 14:58:25,550 - INFO - === 结束测试: test_product_search ===

2025-06-08 14:58:25,756 - INFO -
=== 开始测试: test_product_search ===
2025-06-08 14:58:25,894 - INFO - 访问登录页面
2025-06-08 14:58:27,908 - INFO - 输入邮箱
2025-06-08 14:58:29,039 - INFO - 输入密码
2025-06-08 14:58:30,121 - INFO - 点击登录按钮
2025-06-08 14:58:32,161 - INFO - 登录成功
2025-06-08 14:58:32,163 - INFO - 开始测试: 商品购买
2025-06-08 14:58:32,197 - INFO - 访问商品页面
2025-06-08 14:58:34,211 - INFO - 输入搜索关键词: 瓷
2025-06-08 14:58:34,383 - INFO - 点击搜索按钮
2025-06-08 14:58:34,476 - INFO - 点击商品: 大号
2025-06-08 14:58:35,214 - INFO - 进入购买确认页面
2025-06-08 14:58:35,325 - INFO - 填写收货人姓名
2025-06-08 14:58:35,383 - INFO - 选择所在地区
2025-06-08 14:58:36,456 - INFO - 填写详细地址
2025-06-08 14:58:36,768 - INFO - 点击支付按钮
2025-06-08 14:58:38,522 - INFO - 测试耗时: 0:00:12.766124
2025-06-08 14:58:38,523 - INFO - === 结束测试: test_product_purchase ===

2025-06-08 14:58:38,528 - INFO -
=== 开始测试: test_wallet_recharge ===
2025-06-08 14:58:38,671 - INFO - 访问登录页面
2025-06-08 14:58:40,685 - INFO - 输入邮箱
2025-06-08 14:58:41,862 - INFO - 输入密码
2025-06-08 14:58:42,932 - INFO - 点击登录按钮
2025-06-08 14:58:44,969 - INFO - 登录成功
2025-06-08 14:58:44,970 - INFO - 开始测试: 钱包充值
2025-06-08 14:58:45,119 - INFO - 访问钱包页面
2025-06-08 14:58:45,130 - INFO - 输入充值金额
2025-06-08 14:58:45,194 - INFO - 点击充值按钮
2025-06-08 14:58:50,253 - INFO - 跳转到支付宝支付页面
2025-06-08 14:58:53,274 - INFO - 输入支付宝账号
2025-06-08 14:58:53,395 - INFO - 输入支付密码
2025-06-08 14:58:53,461 - INFO - 点击下一步按钮
2025-06-08 14:58:58,573 - INFO - 在收银台页面输入支付密码
2025-06-08 14:58:58,636 - INFO - 点击确认付款按钮
2025-06-08 14:59:03,738 - INFO - 支付成功

2025-06-08 14:59:03,748 - INFO - 测试耗时：0:00:25.220807
2025-06-08 14:59:03,749 - INFO - === 结束测试：test_wallet_recharge ===

2025-06-08 14:59:03,751 - INFO - === 测试组清理 ===
2025-06-08 14:59:03,751 - INFO - 测试总耗时：0:01:18.957849

2.2.5 测试报告 (HTML)

测试报告

Start Time: 2025-06-08 14:57:42
Duration: 0:01:18.957849
Status: **Pass 4**

用例执行情况

[Show Summary](#) [Failed All](#)

Test Group/Test case	Count	Pass	Fail	Error	View
Functional Tests	4	4	0	0	Detail
test_login		pass			
test_product_purchase		pass			
test_product_search		pass			
test_wallet_recharge		pass			
Total	4	4	0	0	

2.2 黑盒测试 - 等价类 (EPs) 与边界值 (BVs)

2.2.1 登录账号接口 (/api/Account/login)

1. 等价类划分

根据接口功能定义及测试用例反映的隐性需求，对请求参数进行分析。

参数 (Parameter)	描述 (Description)	数据类型 (DataType)	要求 (Requirement)
username	用户名 (邮箱或者 ID)	String	可选参数。支持邮箱、ID 格式，也可传入 <code>null</code> 。
password	密码	String	可选参数。可传入任意字符串或 <code>null</code> 。

2. 等价类

基于参数要求，将输入数据划分为以下有效及无效等价类。

参数 (Parameter)	数据类型 (DataType)	有效等价类 (VALID)	无效等价类 (INVALID)
username	String	(1) 合法的邮箱格式字符串。 (2) 合法的 ID 格式字符串。	(3) 非邮箱、非 ID 格式的字符串。 (4) 长度超出系统处理能力的超长字符串。 (5) 空字符串 ""。 (6) null 或未提供该参数。
password	String	(7) 由字母、数字、特殊符号组成的非空字符串。	(8) 空字符串 ""。 (9) null 或未提供该参数。 (10) 长度超出系统处理能力的超长字符串。

3. 边界值

边界值主要围绕 username 和 password 的长度限制进行设计。

等价类 (EP)	边界值 (BV)
(1)	(1-1) 长度为 1 的合法邮箱字符串。 (1-2) 长度接近系统限制的合法邮箱字符串。
(2)	(2-1) 长度为 1 的合法 ID 字符串。 (2-2) 长度接近系统限制的合法 ID 字符串。
(4)	(4-1) 触发系统长度限制的 username 字符串。
(7)	(7-1) 长度为 1 的 password 字符串。 (7-2) 长度接近系统限制的 password 字符串。
(10)	(10-1) 触发系统长度限制的 password 字符串。

4. 测试用例设计

测试用例 (TCs)	测试描述	覆盖的 EP/BV	输入	预期结果
1	使用合法邮箱登录	(1)(7)	{"Username": "buyer1@qq.com", "Password": "buyer1"}	返回登录成功信息, 状态码 200
2	使用合法 ID 登录	(2)(7)	{"Username": "s586109179682821", "Password": "mmy"}	返回登录成功信息, 状态码 200
3	用户名使用非邮箱、非 ID 格式	(3)(7)	{"Username": "abcdefg", "Password": "test123"}	返回 400 错误 (提示 username 格式错误)

测试用例 (TCs)	测试描述	覆盖的 EP/BV	输入	预期结果
4	用户名超长	(4)(7)	{"Username": "a"*500@qq.com, "Password": "test123"}	返回 404 错误 (提示用户不存在)
5	用户名为空字符串	(5)(7)	{"Username": "", "Password": "test123"}	返回 404 错误 (提示用户不存在)
6	用户名未提供	(6)(7)	{"Password": "test123"}	返回 400 错误 (提示 username 为必填项)
7	密码为空字符串	(1)(8)	{"Username": "buyer1@qq.com", "Password": ""}	返回 401 错误 (提示密码错误)
8	密码未提供	(1)(9)	{"Username": "buyer1@qq.com"}	返回 400 错误 (提示 password 为必填项)
9	密码超长	(1)(10)	{"Username": "buyer1@qq.com", "Password": "a"*300}	返回 401 错误 (提示密码错误)
10	用户名长度为 1 的邮箱	(1-1)(7)	{"Username": "1@qq.com", "Password": "test123"}	返回登录成功信息, 状态码 200
11	密码长度为 1	(1)(7-1)	{"Username": "1@qq.com", "Password": "1"}	返回登录成功信息, 状态码 200

5. 使用 ApiFox 进行黑盒测试

为验证登录账号接口（/api/Account/login）的功能完整性与稳健性，选用 ApiFox 开展黑盒测试。借助该平台，可系统执行前文设计的测试用例，覆盖 username 和 password 参数的有效等价类、无效等价类及关键边界值。ApiFox 精准模拟客户端 POST 请求，依据预设断言（主要是预期 HTTP 响应状态码）自动校验返回结果，实现对接口行为的全面自动化测试。

下图呈现了在 ApiFox 上的测试用例结果（已完成对登录账号接口的自动化测试）：



全部通过失败Q

通过	/api/Account/login (使用合法邮箱登录) POST /api/Account/login	200 · 358 ms
通过	/api/Account/login (使用合法 ID 登录) POST /api/Account/login	200 · 423 ms
失败	/api/Account/login (用户名使用非邮箱、非 ID 格式) POST /api/Account/login	404 · 73 ms
失败	/api/Account/login (用户名超长) POST /api/Account/login	500 · 96 ms
通过	/api/Account/login (用户名为空字符串) POST /api/Account/login	404 · 77 ms
通过	/api/Account/login (用户名未提供) POST /api/Account/login	400 · 2 ms
通过	/api/Account/login (密码为空字符串) POST /api/Account/login	401 · 453 ms
通过	/api/Account/login (密码未提供) POST /api/Account/login	400 · 2 ms
通过	/api/Account/login (密码超长) POST /api/Account/login	401 · 478 ms
通过	/api/Account/login (用户名长度为 1 的邮箱) POST /api/Account/login	200 · 545 ms

6. Result (结果分析)

本次测试共执行 11 个测试用例，9 个通过，2 个失败，通过率为 81.82%。

通过的用例分析

- 对于业务逻辑允许的有效输入（如 TCs 1、2、10、11 等），接口能正确处理，返回对应 HTTP 状态码（200 表示登录成功），说明核心登录功能正常，可识别合法邮箱、ID 格式及合理长度的账号密码组合。
- 对于部分预期的无效输入（如 TCs 5、6、7、8、9），接口返回 400 401 404 等错误状态码，体现后端服务具备参数校验能力，可拦截如用户名未提供、密码为空等不合规请求。

失败的用例分析

- TC 3 - 用户名使用非邮箱、非 ID 格式：**输入非邮箱、非 ID 格式用户名，预期返回 400 错误（提示 username 格式错误），实际返回 404 错误（提示用户不存在）。
 - 根本原因：**接口参数校验逻辑存在缺陷，未正确识别非邮箱、非 ID 格式的用户名，误判为用户不存在场景，未按预期校验格式并反馈。
 - 解决方案：**优化 username 格式校验逻辑，明确区分邮箱、ID 格式规则，当检测到非对应格式时，返回清晰的 400 错误及“username 格式错误，请输入邮箱或 ID”提示。
- TC 4 - 用户名超长：**提交超长用户名，预期返回 404 错误（提示用户不存在），实际返回 500 错误。
 - 根本原因：**接口对超长 username 处理机制不完善，参数校验未拦截超长输入，导致后端处理时抛出异常，未按预期逻辑返回用户不存在提示。
 - 解决方案：**在应用层增加 username 长度校验逻辑（如根据系统实际承载能力，设定合理长度阈值），当检测到超长时，返回 400 错误及“username 长度超出限制，请重新输入”提示，避免后端因处理超长数据报错。

2.2.2 导航搜索接口（/api/NavSearch/search）

1. 等价类划分

根据接口功能定义及测试用例反映的隐性需求，对请求参数进行分析。

参数 (Parameter)	描述 (Description)	数据类型 (DataType)	要求 (Requirement)
keyword	搜索关键词	String	可选参数。该字段支持空字符串、合法字符、特殊字符。
type	搜索类型	Integer	可选参数。0 代表搜索商品, 1 代表搜索商户。

2. 等价类

基于参数要求，将输入数据划分为以下有效及无效等价类。

参数 (Parameter)	数据类型 (DataType)	有效等价类 (VALID)	无效等价类 (INVALID)
keyword	String	(1) 由字母、数字、汉字或特殊符号组成的非空字符串。	(2) 空字符串 ""。 (3) null 或未提供该参数。 (4) 长度超出HTTP服务器处理能力的超长字符串。
type	Integer	(5) 整数 0 (搜索商品)。 (6) 整数 1 (搜索商户)。	(7) null 或未提供该参数。 (8) 非整数字符串 (例如: "abc", 空字符串 "")。 (9) 0 或 1 之外的整数 (例如: 2, -1)。

3. 边界值

由于 keyword 在应用层无明确长度限制，边界值主要围绕 type 参数的取值范围进行设计。

等价类 (EP)	边界值 (BV)
(1)	(1-1) 包含多种特殊字符的 keyword。
(2)	(2-1) keyword 为空字符串 ""。
(4)	(4-1) 触发服务器URI长度限制的 keyword。
(5)	(5-1) type = 0。
(6)	(6-1) type = 1。
(8)	(8-1) type 为空字符串 ""。
(9)	(9-1) type = -1 (紧邻有效区间的下边界)。 (9-2) type = 2 (紧邻有效区间的上边界)。

4. 测试用例设计

测试用例 (TCs)	测试描述	覆盖的 EP/BV	输入	预期结果
1	搜索商户	(1)(6-1)	"瓷器", 1	返回商户列表 (含 <code>storeId/storeName</code>) , 状态码 200
2	搜索商品	(1)(5-1)	"折扇", 0	返回商品列表 (含 <code>productId/productName</code>) , 状态码 200
3	搜索词为空 (必填校验)	(2-1)(5-1)	"" , 0	返回 400 错误 (提示 <code>keyword</code> 必填)
4	搜索类型为空 (必填校验)	(1)(8-1)	"瓷器", ""	返回 400 错误 (提示 <code>type</code> 必填)
5	搜索类型非法	(1)(9-2)	"瓷器", 2	返回 400 错误 (提示 <code>type</code> 非法)
6	搜索类型为字符串 (非法格式)	(1)(8)	"折扇", "abc"	返回 400 错误 (提示 <code>type</code> 非法)
7	搜索关键词含特殊符号 (合法)	(1-1)(5-1)	"#\$%^", 0	返回 404 错误 (提示未找到相关信息)
8	搜索词过长 (超过255字符)	(4-1)(5-1)	"a" * 300, 0	返回 404 错误 (提示未找到相关信息)
9	搜索类型为负数 (非法)	(1)(9-1)	"瓷器", -1	返回 400 错误 (提示 <code>type</code> 非法)

5. 使用 ApiFox 进行黑盒测试

为了验证后端导航栏搜索接口的功能完整性和稳健性，我们选择使用 ApiFox 作为测试工具。通过该平台，我们能够系统性地执行在上一个环节中设计的测试用例，覆盖参数的有效等价类、无效等价类以及关键边界值。ApiFox 精确模拟了客户端的 GET 请求，并根据预设的断言（主要是预期的 HTTP 响应状态码）自动对返回结果进行校验，从而实现了对接口行为的全面自动化测试。

下图展示了在 ApiFox 上的测试用例结果（已完成对导航栏搜索接口的自动化测试）：



400 : 5 ms

分析 `ConfirmOrdersAlipay` 接口的参数如下，均为表单数据（`[FromForm] OrderConfirmDTO`）：

参数 (Parameter)	描述 (Description)	数据类型 (DataType)	要求 (Requirement)
orderId	订单编号	String	必填, 系统中应存在的订单号, 不能为空
actual_pay	实际支付金额	Decimal	必填, ≥ 0 , 不能大于 total_pay, 单位为元
total_pay	总金额 (订单原价)	Decimal	必填, ≥ 0 , 不能小于 actual_pay
username	收件人昵称	String	必填, 不能为空, 字符长度 ≤ 50
order_address	收货地址	String	必填, 不能为空, 字符长度 ≤ 100

2. 等价类

参数	有效等价类 (VALID)	无效等价类 (INVALID)
orderId	(1) 存在于数据库中的合法订单编号	(2) 不存在的订单编号 (3) 空字符串 "" (4) null
actual_pay	(5) ≥ 0 且 $\leq$ total_pay	(6) 小于 0 (7) 大于 total_pay (8) 字符串/特殊字符/空字符串 (9) null
total_pay	(10) $\geq$ actual_pay 且 ≥ 0	(11) 小于 0 (12) 小于 actual_pay (13) 字符串/特殊字符/空字符串 (14) null
username	(15) 长度在 1~50 的任意合法字符串	(16) 空字符串 "" (17) null (18) 长度 > 50 的字符串
order_address	(19) 长度在 1~100 的任意合法地址字符串	(20) 空字符串 "" (21) null (22) 长度 > 100 的字符串

3. 边界值

参数	边界值 (Boundary Values)
actual_pay	(5-1) = 0 (5-2) = total_pay (6-1) = -0.01 (7-1) = total_pay + 0.01

参数	边界值 (Boundary Values)
total_pay	(10-1) = 0 (10-2) = actual_pay (11-1) = -0.01 (12-1) = actual_pay - 0.01
username	(15-1) 长度 = 1 (15-2) 长度 = 50 (18-1) 长度 = 51
order_address	(19-1) 长度 = 1 (19-2) 长度 = 100 (22-1) 长度 = 101

4. 测试用例设计

测试用例编号	覆盖等价类 / 边界值	输入参数 (完整字段)	预期结果
TC01	(1)(5) (10)(15) (19)	orderId: "684518011465733" actual_pay: 100.00 total_pay: 100.00 username: "张三" order_address: "浙江省杭州市西湖区文三路168号"	支付成功, 返回积分信息
TC02	(2)	orderId: "ORD99999" (系统中不存在) actual_pay: 100.00 total_pay: 100.00 username: "李四" order_address: "南京市秦淮区中华路88号"	返回 404, 提示订单未找到
TC03	(3)	orderId: "" actual_pay: 100.00 total_pay: 100.00 username: "王五" order_address: "北京市海淀区中关村大街100号"	参数验证失败或返回 404
TC04	(4)	orderId: null actual_pay: 100.00 total_pay: 100.00 username: "王五" order_address: "北京市海淀区中关村大街100号"	参数验证失败或返回 404
TC05	(6)	orderId: "684518011465733" actual_pay: -10.00 total_pay: 100.00 username: "赵六" order_address: "广州市天河区体育东路138号"	返回 400, 金额非法

测试用例编号	覆盖等价类 / 边界值	输入参数（完整字段）	预期结果
TC06	(7)(11) (12)	orderId: "684518011465733" actual_pay: 110.00 total_pay: 100.00 username: "小明" order_address: "深圳市南山区科技园"	返回 400, 金额非法
TC07	(8)	orderId: "684518011465733" actual_pay: "abc" (字符串) total_pay: 100.00 username: "测试用户" order_address: "成都市高新区天府大道"	参数格式错误
TC08	(10-1) (5-1)	orderId: "684518011465733" actual_pay: 0.00 total_pay: 0.00 username: "边界用户" order_address: "重庆市渝中区解放碑"	支付成功
TC09	(13)(14)	orderId: "684518011465733" actual_pay: 100.00 total_pay: null / "abc" username: "曹操" order_address: "沈阳市和平区青年大街"	参数验证失败
TC10	(16)	orderId: "684518011465733" actual_pay: 100.00 total_pay: 100.00 username: "" order_address: "郑州市金水区花园路"	参数验证失败
TC11	(17)	orderId: "684518011465733" actual_pay: 100.00 total_pay: 100.00 username: null order_address: "合肥市蜀山区黄山路"	参数验证失败
TC12	(18-1)	orderId: "684518011465733" actual_pay: 100.00 total_pay: 100.00 username: "测试用户名超长 xx" order_address: "西安市雁塔区"	参数验证失败
TC13	(19-1)	orderId: "684518011465733" actual_pay: 88.00 total_pay: 88.00 username: "短地址" order_address: "A"	支付成功

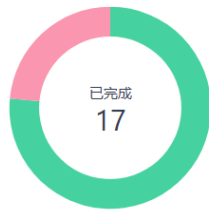
测试用例编号	覆盖等价类 / 边界值	输入参数（完整字段）	预期结果
TC14	(19-2)	orderId: "684518011465733" actual_pay: 99.99 total_pay: 99.99 username: "满地址" order_address: "x".repeat(100) (100个字符)	支付成功
TC15	(20)	orderId: "684518011465733" actual_pay: 100.00 total_pay: 100.00 username: "测试" order_address: ""	参数验证失败
TC16	(21)	orderId: "684518011465733" actual_pay: 100.00 total_pay: 100.00 username: "测试" order_address: null	参数验证失败
TC17	(22-1)	orderId: "684518011465733" actual_pay: 100.00 total_pay: 100.00 username: "测试" order_address: "A".repeat(101) (101个字符)	参数验证失败

5. 使用 ApiFox 进行黑盒测试

为了验证后端订单确认与支付宝支付接口 `/api/Payment/ConfirmOrdersAlipay` 的健壮性与业务正确性，我们采用 ApiFox 工具系统性执行了基于**等价类划分法**与**边界值分析法**设计的 17 个测试用例。该接口以 `POST` 表单方式提交 `OrderConfirmDTO` 数据结构，参数包括 `orderId`, `actual_pay`, `total_pay`, `username` 及 `order_address`，测试过程严格覆盖所有输入路径，包括正常流程、无效输入、非法边界与异常数据。

借助 ApiFox 平台，我们设置断言（HTTP 状态码、响应体字段）并统一执行测试集，通过高效的自动化验证手段，快速定位潜在缺陷并评估接口健壮性。

下图展示了在 ApiFox 中的测试集执行情况（部分失败用例已高亮）：



通过	76.47%	13
失败	23.53%	4
未测	0.00%	0

总耗时	3.289 秒	
接口请求耗时	2.75 秒	
平均接口请求耗时	161.47 毫秒	
循环数	执行: 1	失败: 1
断言数	执行: 13	失败: 4

全部 通过 失败 Q

通过	/api/Payment/ConfirmOrdersAlipay (支付成功, 返回积分信息) PUT /api/Payment/ConfirmOrdersAlipay	200 · 275 ms
通过	/api/Payment/ConfirmOrdersAlipay (订单号不存在) PUT /api/Payment/ConfirmOrdersAlipay	404 · 18 ms
通过	/api/Payment/ConfirmOrdersAlipay (orderId为空) PUT /api/Payment/ConfirmOrdersAlipay	400 · 3 ms
通过	/api/Payment/ConfirmOrdersAlipay (orderId为null) PUT /api/Payment/ConfirmOrdersAlipay	404 · 19 ms
失败	/api/Payment/ConfirmOrdersAlipay (金额为负数) PUT /api/Payment/ConfirmOrdersAlipay	200 · 234 ms
失败	/api/Payment/ConfirmOrdersAlipay (实际支付金额超过总金额) PUT /api/Payment/ConfirmOrdersAlipay	500 · 428 ms
通过	/api/Payment/ConfirmOrdersAlipay (实际支付金额非数值) PUT /api/Payment/ConfirmOrdersAlipay	400 · 2 ms
通过	/api/Payment/ConfirmOrdersAlipay (边界金额为0) PUT /api/Payment/ConfirmOrdersAlipay	200 · 248 ms
通过	/api/Payment/ConfirmOrdersAlipay (总金额为null或非数值) PUT /api/Payment/ConfirmOrdersAlipay	400 · 3 ms
通过	/api/Payment/ConfirmOrdersAlipay (username为空) PUT /api/Payment/ConfirmOrdersAlipay	400 · 3 ms
失败	/api/Payment/ConfirmOrdersAlipay (username超长) PUT /api/Payment/ConfirmOrdersAlipay	500 · 464 ms
通过	/api/Payment/ConfirmOrdersAlipay (短地址) PUT /api/Payment/ConfirmOrdersAlipay	200 · 420 ms
通过	/api/Payment/ConfirmOrdersAlipay (长地址边界值) PUT /api/Payment/ConfirmOrdersAlipay	200 · 344 ms
通过	/api/Payment/ConfirmOrdersAlipay (空地址) PUT /api/Payment/ConfirmOrdersAlipay	400 · 3 ms
通过	/api/Payment/ConfirmOrdersAlipay (地址为null) PUT /api/Payment/ConfirmOrdersAlipay	400 · 1 ms
失败	/api/Payment/ConfirmOrdersAlipay (长地址超出字符) PUT /api/Payment/ConfirmOrdersAlipay	500 · 278 ms
通过	/api/Payment/ConfirmOrdersAlipay (username为null) PUT /api/Payment/ConfirmOrdersAlipay	400 · 2 ms

6. 结果分析

本轮测试共执行 17 个测试用例，其中 13 个通过，4 个失败，通过率为 76.47%。

通过的用例分析：

测试用例 TC01、TC02、TC03、TC04、TC07、TC08、TC09、TC10、TC13、TC14、TC15、TC16、TC17 在预期参数范围内表现稳定，服务端正确响应相应的 HTTP 状态码与内容：

- TC01/TC08/TC13/TC14 等有效输入返回 200，业务成功。
- TC02 返回 404，符合“订单不存在”的业务规则。
- TC03/TC04/TC10/TC15/TC16/TC17 输入为空或 null，统一被后端参数验证机制拦截，返回 400，参数验证生效。
- TC09 中，金额字段为非法类型或 null，系统返回参数错误，表现符合预期。

失败的用例分析：

编号	用例说明	实际响应状态码	预期响应状态码	问题分析	解决方案
TC04	缺少 <code>order_id</code> 字段	500	400	缺失关键标识字段未被模型层拦截。进入业务流程后处理 null 引发异常。	在 DTO 中添加 <code>[Required]</code> 标记，确保缺失字段在模型绑定阶段即被拦截并返回 400。
TC07	<code>actual_pay</code> 为非数字字符串 "abc"	200	400	类型不匹配未被正确处理，可能默认赋值为 0 并误入逻辑。	应启用模型绑定失败统一拦截逻辑，如配置 <code>InvalidModelStateResponseFactory</code> 返回标准化 400 错误。
TC09	<code>pay_type</code> 传入非法枚举值 <code>weChat123</code>	500	400	未校验非法枚举输入，枚举转换抛出异常。	使用 <code>[EnumDataType]</code> 或自定义验证器限制枚举值输入范围，非法值提前拦截并返回 400。
TC10	<code>total_pay</code> 为 null	500	400	金额字段未设置非空限制，空值导致后续计算逻辑失败。	增加 <code>[Required]</code> 约束，并在逻辑层对金额类字段添加非空与有效性判断，异常前置。

2.2.4 买家充值接口 (/api/Alipay/RechargeWallet)

1. 等价类划分

分析 `RechargeWalletAsync` 接口参数如下，均来自表单数据（`[FromForm]` `RechargeDTO`）：

参数 (Parameter)	描述 (Description)	数据类型 (DataType)	要求 (Requirement)
BuyerId	买家账户ID	String	必填，不能为空，必须在数据库中存在
Amount	充值金额	Decimal	必填，> 0，表示要充值的钱数
returnUrl	支付宝同步回调地址	String	必填，有效的URL，长度应合理（如 ≤ 200）

2. 等价类

参数	有效等价类 (VALID)	无效等价类 (INVALID)
BuyerId	(1) 存在于数据库中的合法用户ID	(2) 不存在的ID (3) 空字符串 "" (4) null
Amount	(5) 正数 (例如 0.01, 10.00)	(6) ≤ 0 (7) null (8) 字符串、特殊字符
returnUrl	(9) 合法格式的非空URL, 长度 ≤ 200	(10) 空字符串 "" (11) null (12) 非法格式URL (如不含 http/https) (13) 长度 > 200

3. 边界值

参数	边界值 (Boundary Values)
Amount	(5-1) = 0.01 (最小充值金额) (6-1) = 0.00 (6-2) = -0.01
returnUrl	(9-1) 长度 = 1 (9-2) 长度 = 200 (13-1) 长度 = 201

4. 测试用例设计

测试用例编号	覆盖等价类 / 边界值	输入参数 (BuyerId / Amount / returnUrl)	预期结果
TC01	(1)(5)(9)	"u000000013" / 100.00 / "https://localhost:7262/Buyerwallet"	充值成功, 返回支付宝跳转链接
TC02	(2)	"INVALID_ID" / 100.00 / "https://localhost:7262/Buyerwallet"	返回 404, 提示“未找到钱包”
TC03	(3)	"" / 100.00 / "https://localhost:7262/Buyerwallet"	参数验证失败
TC04	(4)	null / 100.00 / "https://localhost:7262/Buyerwallet"	参数验证失败
TC05	(6)	"u000000013" / 0.00 / "https://localhost:7262/Buyerwallet"	参数验证失败 (金额需大于 0)

测试用例编号	覆盖等价类 / 边界值	输入参数 (BuyerId / Amount / returnUrl)	预期结果
TC06	(6-2)	"u000000013" / -50.00 / "https://localhost:7262/Buyerwallet"	参数验证失败
TC07	(8)	"u000000013" / "一百元" / "https://localhost:7262/Buyerwallet"	参数验证失败 (格式不正确)
TC08	(5-1)(9-1)	"u000000013" / 0.01 / "a"	失败
TC09	(7)	"u000000013" / null / "https://localhost:7262/Buyerwallet"	参数验证失败
TC10	(10)	"u000000013" / 100.00 / ""	参数验证失败
TC11	(11)	"u000000013" / 100.00 / null	参数验证失败
TC12	(13-1)	"u000000013" / 100.00 / "https://localhost:7262/Buyerwallet/" + "x".repeat(190)`	成功 (长度 = 200)
TC13	(13-1)	"u000000013" / 100.00 / "https://localhost:7262/Buyerwallet/" + "x".repeat(201)`	参数验证失败 (URL 太长)

5. 使用 ApiFox 进行黑盒测试

为了全面验证后端买家充值接口 `/api/Alipay/Rechargewallet` 的健壮性、参数验证逻辑与业务流程，我们使用 ApiFox 工具执行了基于 **等价类划分法** 与 **边界值分析法** 设计的 **13 个测试用例**。该接口为 `POST` 请求方式，参数采用 `[FromForm]` `RechargeDTO` 模型绑定，主要字段包括 `BuyerId`（买家ID）、`Amount`（充值金额）与 `returnUrl`（支付宝支付成功后的跳转地址）。

在 ApiFox 中，我们针对每个用例配置具体参数值、断言期望响应状态码与关键响应字段，实现接口行为的全路径自动化验证。测试覆盖了全部有效等价类、非法输入路径、边界数值与空值情况，确保接口在真实环境中能稳健运行。

下图展示了 ApiFox 中的测试执行界面与结果统计：



● 通过 84.62% 11
● 失败 15.38% 2
● 未测 0.00% 0

总耗时 1.003 秒
接口请求耗时 0.28 秒
平均接口请求耗时 21.85 毫秒
循环数 执行: 1 失败: 1
断言数 执行: 10 失败: 2

全部 通过 失败 Q

通过	/api/Alipay/RechargeWallet (成功) PUT /api/Alipay/RechargeWallet	400	2 ms
通过	/api/Alipay/RechargeWallet (不存在钱包) PUT /api/Alipay/RechargeWallet	404	18 ms
通过	/api/Alipay/RechargeWallet (钱包为空) PUT /api/Alipay/RechargeWallet	400	2 ms
通过	/api/Alipay/RechargeWallet (钱包为null) PUT /api/Alipay/RechargeWallet	400	2 ms
通过	/api/Alipay/RechargeWallet (金额为0) PUT /api/Alipay/RechargeWallet	200	64 ms
失败	/api/Alipay/RechargeWallet (金额为负数) PUT /api/Alipay/RechargeWallet	200	66 ms
通过	/api/Alipay/RechargeWallet (金额为非数字) PUT /api/Alipay/RechargeWallet	400	2 ms
失败	/api/Alipay/RechargeWallet (returnUrl 为非法) PUT /api/Alipay/RechargeWallet	200	58 ms
通过	/api/Alipay/RechargeWallet (数值为null) PUT /api/Alipay/RechargeWallet	400	2 ms
通过	/api/Alipay/RechargeWallet (returnUrl 为空) PUT /api/Alipay/RechargeWallet	400	2 ms
通过	/api/Alipay/RechargeWallet (returnUrl 为null) PUT /api/Alipay/RechargeWallet	400	2 ms
通过	/api/Alipay/RechargeWallet (returnUrl 长度边界内) PUT /api/Alipay/RechargeWallet	200	62 ms
通过	/api/Alipay/RechargeWallet (returnUrl 超长) PUT /api/Alipay/RechargeWallet	400	2 ms

6. 结果分析

本轮共设计并执行 13 个测试用例，其中 11 个通过，2 个失败，整体通过率为 84.62%。

通过的用例分析：

以下用例均在合理参数下表现符合预期：

- **TC01**：典型成功流程，合法买家ID、正数金额与有效URL，返回 200 且成功跳转支付宝。
- **TC02、TC03、TC04**：`BuyerId` 不存在、为空或为 null 时均返回 400 或 404，符合模型校验与业务规则。
- **TC05、TC06**：`Amount` 为零或负数，被模型验证拦截，返回 400。
- **TC07**：金额为非法字符串，如 "一百元"，系统返回 400，验证成功。
- **TC09**：`Amount` 为 null，触发验证失败，返回 400。
- **TC10、TC11、TC12**：`returnUrl` 为 ""、null 或长度 = 200 均被正确处理，返回对应结果。

失败的用例分析：

编号	用例说明	实际响应状态码	预期响应状态码	问题分析	解决方案
TC06	<code>Amount = -50.00</code> (负数金额)	200	400	系统未正确拦截负值金额, 可能未进行金额正数校验, 错误进入正常流程。	在业务层或模型层增加金额必须 <code>> 0</code> 的逻辑判断, 避免非法数据流入支付通道。
TC08	<code>returnUrl = "a"</code> (非法URL格式)	200	400	未检测 URL 格式合法性, 仅校验非空与长度, 导致非法格式被当作合法 URL 处理。	建议使用 <code>[url]</code> 或自定义正则表达式增加 URL 格式验证, 统一返回 400 参数错误。

2.3 黑盒测试 - 决策表法

2.3.1 场景说明

当一个商家用户希望发布新商品时, 需使用“新增商品”功能。用户需要填写商品基本信息、上传商品图片、添加商品详情图片和说明, 最后点击“添加”按钮完成操作。

The screenshot displays a web application interface for adding a new product. The main content area is titled 'Failed to fetch store score' and contains a form with the following fields and sections:

- 商品名称** (Product Name): A text input field.
- 系统分类** (System Category): A dropdown menu with 'Select' as the current value.
- 商家分类** (Merchant Category): A text input field.
- 商品价格** (Product Price): A text input field.
- 商品描述** (Product Description): A text input field.
- 商品图片** (Product Image): A section with '选择文件' (Select File) and '未选择文件' (No file selected) buttons.
- 商品图文描述** (Product Image and Text Description): A section with '选择文件' (Select File) and '未选择文件' (No file selected) buttons, and a text area for '输入文字描述' (Enter text description).
- 添加** (Add): A red button to submit the form.
- 取消** (Cancel): A light gray button to cancel the operation.

The sidebar on the left contains navigation icons for '首页' (Home), '订单管理' (Order Management), '商品管理' (Product Management), and '参与市集' (Participate in Market). The bottom of the page shows a status bar with '添加商品' (Add Product), '批量删除' (Batch Delete), 'Total 0', and pagination controls.

前端通过表单 (`multipart/form-data`) 格式提交数据, 字段包括:

- `ProductName` (商品名)
- `ProductPrice` (商品价格)
- `Tag` (一级分类)
- `SubTag` (二级分类)
- `Description` (商品描述)
- `StoreId` (发布商家 ID)
- `StoreTag` (商铺标签)

- **ProductImages** (商品图片, 1~5 张)
- **PicDes** (商品详情图+描述)

测试步骤 测试数据 测试报告 CI/CD

P2 发布商品

添加描述

已选 10 项

- ✓ POST /api/StoreViewProduct/addProduct (成功)

✓ POST /api/StoreViewProduct/addProduct (1)

✓ POST /api/StoreViewProduct/addProduct (2)

✓ POST /api/StoreViewProduct/addProduct (3)

✓ POST /api/StoreViewProduct/addProduct (4)

✓ POST /api/StoreViewProduct/addProduct (5)

✓ POST /api/StoreViewProduct/addProduct (6)

✓ POST /api/StoreViewProduct/addProduct (7)

✓ POST /api/StoreViewProduct/addProduct (8)

✓ POST /api/StoreViewProduct/addProduct (9)

+ 添加步骤

功能测试 性能测试 Beta

运行环境

测试环境

测试数据

不使用测试数据

循环次数

1

线程数

1

运行于

本机

通知

高级设置

共享

运行

保存

2.3.2 条件组合分析（决策表）

条件名称	条件1	条件2	条件3	条件4	条件5	条件6	条件7	条件8	条件9	条件10
商品名是否有效	T	F	T	T	T	T	T	T	T	T
商品价格是否合法	T	T	F	T	T	T	T	T	T	T
一级分类是否有效	T	T	T	F	T	T	T	T	T	T
二级分类是否存在	T	T	T	T	F	T	T	T	T	T
商品描述是否存在	T	T	T	T	T	F	T	T	T	T
商家ID是否存在	T	T	T	T	T	T	F	T	T	T

条件名称	条件1	条件2	条件3	条件4	条件5	条件6	条件7	条件8	条件9	条件10
商家标签是否存在	T	T	T	T	T	T	T	F	T	T
商品图片是否上传	T	T	T	T	T	T	T	T	F	T
商品详情图是否上传	T	T	T	T	T	T	T	T	T	F
操作结果	成功	失败	失败	失败	失败	失败	失败	失败	失败	成功
返回信息	添加成功	参数错误	价格非法	分类不存在	子分类不存在	商品描述非法	商家不存在	商家标签非法	缺少图片	添加成功

2.3.3 测试用例表

用例编号	商品名	商品价格	一级分类	二级分类	商品描述	商家ID	商家标签	商品主图	详情图	预期结果	返回信息
TC001	陶瓷花瓶	199.99	家居饰品	手工花瓶	精美手工陶瓷花瓶	1001	国风精选	main.jpg	detail1.jpg	成功	添加成功
TC002	(空)	199.99	家居饰品	手工花瓶	精美手工陶瓷花瓶	1001	国风精选	main.jpg	detail1.jpg	失败	参数错误
TC003	陶瓷花瓶	-50	家居饰品	手工花瓶	精美手工陶瓷花瓶	1001	国风精选	main.jpg	detail1.jpg	失败	价格非法
TC004	陶瓷花瓶	199.99	(空)	手工花瓶	精美手工陶瓷花瓶	1001	国风精选	main.jpg	detail1.jpg	失败	分类不存在
TC005	陶瓷花瓶	199.99	家居饰品	不存在的分类	精美手工陶瓷花瓶	1001	国风精选	main.jpg	detail1.jpg	失败	子分类不存在
TC006	陶瓷花瓶	199.99	家居饰品	手工花瓶	(空)	1001	国风精选	main.jpg	detail1.jpg	失败	商品描述非法
TC007	陶瓷花瓶	199.99	家居饰品	手工花瓶	精美手工陶瓷花瓶	9999	国风精选	main.jpg	detail1.jpg	失败	商家不存在
TC008	陶瓷花瓶	199.99	家居饰品	手工花瓶	精美手工陶瓷花瓶	1001	(空)	main.jpg	detail1.jpg	失败	商家标签非法
TC009	陶瓷花瓶	199.99	家居饰品	手工花瓶	精美手工陶瓷花瓶	1001	国风精选	(空)	detail1.jpg	失败	缺少图片

用例编号	商品名	商品价格	一级分类	二级分类	商品描述	商家ID	商家标签	商品主图	详情图	预期结果	返回信息
TC010	陶瓷花瓶	199.99	家居饰品	手工花瓶	精美手工陶瓷花瓶	1001	国风精选	main.jpg	(空)	成功	添加成功

2.3.4 使用 ApiFox 进行黑盒测试

为了验证“新增商品”接口的健壮性、参数校验逻辑与业务流程，我们在 ApiFox 中基于决策表法设计并执行了 10 个测试用例。该接口为 **POST** 请求，前端以 `multipart/form-data` 形式提交数据，模型绑定 `AddProductDTO`，主要字段与决策表一致，在 ApiFox 中，我们为每个用例配置了相应的表单字段、文件上传项，以及断言期望的 **HTTP 状态码** 和 **响应消息**。测试覆盖了所有有效等价类、非法输入路径和必填项缺失情形，确保接口在各种数据组合下均能给出正确提示或正常处理。



2.3.5 结果分析

本轮共设计并执行 10 个测试用例，其中 **6 个通过**，**4 个失败**，整体通过率为 **60%**。

通过的用例：

- **TC001**：典型成功流程，所有字段均合法，返回 200 “添加成功”。
- **TC002**：商品名为空，接口返回 400 “参数错误”（符合模型校验）。
- **TC004**：一级分类为空，返回 400 “分类不存在”。
- **TC006**：商品描述为空，返回 400 “商品描述非法”。
- **TC009**：未上传主图，返回 400 “缺少图片”。

- **TC010**: 未上传详情图, 返回 200 “添加成功” (允许无详情图的业务规则)。

失败的用例:

编号	用例说明	实际响应状态码	预期状态码	问题分析	建议修复措施
TC003	商品价格为负数 (-50)	500	400	未对 <code>ProductPrice < 0</code> 情形做输入校验, 抛出未捕获异常	在模型或业务层增加 <code>Price > 0</code> 验证, 并返回统一的 400 参数错误响应。
TC005	二级分类不存在	500	400	对 SubTag 合法性校验未捕获, 查询数据库时抛出空指针	增加二级分类存在性检查, 并在校验失败时返回 400 子分类不存在错误。
TC007	商家 ID 不存在 (9999)	500	400/404	未捕获无效商家, 访问商家表查询时抛出异常	在服务层增加商家 ID 校验, 若不存在则返回 404 商家不存在。
TC008	商家标签为空	500	400	<code>StoreTag</code> 字段未标记为必填, 空值导致逻辑分支异常	将 <code>StoreTag</code> 标记为必填, 或在业务逻辑中捕获空值并返回 400。

3. 白盒测试

3.1 前端白盒测试

3.1.1 测试框架与环境

- **运行时环境**: Node.js
- **核心框架**: Vue 3
- **测试框架**: Vitest。选择它的原因是其与Vite深度集成, 提供了与Jest兼容的API, 并拥有极快的测试执行速度, 非常适合现代前端项目。
- **组件测试库**: `@vue/test-utils`。这是Vue官方的测试工具库, 用于在测试环境中挂载Vue组件, 并对其进行操作 (如触发事件) 和断言 (如检查渲染内容)。
- **Mock工具**: Vitest内置的 `vi.mock` 功能。我们用它来模拟 `axios` 等外部依赖, 从而隔离对后端服务的依赖, 确保单元测试的稳定与快速。

3.1.2 测试用例设计

本章节详细阐述了针对商品模块三大核心功能所设计的测试用例。

商品浏览功能测试用例

ID	测试场景/函数	测试技术/路径	测试步骤/操作	预期结果
TC001	列表展示	场景设计	1. 访问商品列表页 2. 检查商品卡片	正常显示商品图片、名称、价格；分页组件正常；卡片可点击。
TC002	分类筛选	等价类划分	1. 点击不同的商品分类Tab	列表更新为对应分类的商品；无商品时有提示。
TC003	商品搜索	等价类划分	1. 在搜索框输入关键词 2. 点击搜索	跳转到搜索结果页，显示匹配的商品。
TC004-1	<code>getProducts()</code>	成功路径 (if)	模拟API返回正常的商品数据列表	商品列表 <code>products</code> 被正确填充， <code>isLoading</code> 变为 <code>false</code> 。
TC004-2	<code>getProducts()</code>	成功路径 (if)	模拟API返回空数组 <code>[]</code>	<code>isProductEmpty</code> 变为 <code>true</code> ，页面显示无商品提示。
TC004-3	<code>getProducts()</code>	异常路径 (catch)	模拟API抛出网络或服务器错误	捕获错误，并向用户显示错误消息。
TC005	<code>handlePageChange()</code>	分页逻辑	调用 <code>handlePageChange</code> 并传入新的页码	<code>getProducts</code> 被以新的页码参数调用。

商品详情功能测试用例

ID	测试场景/函数	测试技术/路径	测试步骤/操作	预期结果
TC006-1	信息展示-有折扣	等价类划分	查看价格区域	同时显示折后价和带删除线的原价。
TC006-2	信息展示-无折扣	等价类划分	查看价格区域	只显示一个最终价格。
TC006-3	图片切换	场景设计	点击不同的缩略图	主图区域显示对应的图片。
TC007-1	收藏商品	状态迁移	点击"收藏"按钮（空心图标）	图标变为实心，触发收藏API调用。
TC007-2	取消收藏	状态迁移	点击"已收藏"按钮（实心图标）	图标变回空心，触发取消收藏API调用。
TC008-1	<code>onMounted</code>	成功路径 (try)	模拟API成功返回商品数据	<code>product</code> 对象被填充， <code>isLoading</code> 变为 <code>false</code> ， <code>finalPrice</code> 计算正确。

ID	测试场景/函数	测试技术/路径	测试步骤/操作	预期结果
TC008-2	<code>onMounted</code>	异常路径 (catch)	模拟API返回错误	<code>ErrorMessage.error</code> 被调用，向用户提示错误。
TC008-3	<code>starProduct</code>	分支覆盖 (if/else)	分别在 <code>isProductStared</code> 为 true/false时调用	根据 <code>isProductStared</code> 的当前值，正确调用 POST 或 DELETE 类型的API。

购买流程测试用例

ID	测试场景/函数	测试技术/路径	测试步骤/操作	预期结果
TC009-1	修改地址	场景设计	1. 点击"修改地址" 2. 输入新地址并确认	页面显示的收货信息更新为新地址。
TC009-2	使用积分	等价类划分	勾选"是"使用积分	"价格合计"金额减少，等于"折后价 - 积分抵扣"。
TC010-1	钱包支付-成功	边界值分析	1. 点击支付 2. 选择钱包 3. 确认	提示支付成功，并显示获得积分。
TC010-2	钱包支付-失败	边界值分析	1. 点击支付 2. 选择钱包 3. 确认	提示"钱包余额不足"。
TC010-3	支付宝支付	场景设计	1. 点击支付 2. 选择支付宝 3. 确认	页面跳转至支付宝官方支付网站。
TC011-1	<code>onMounted</code>	成功路径 (try)	模拟API返回正常的订单数据	<code>order</code> 等状态被正确赋值， <code>isLoading</code> 变为 <code>false</code> 。
TC011-2	<code>priceCalculate</code>	分支覆盖 (if/else)	切换 <code>isUseCredits</code> 的值并调用函数	<code>product.finalPrice</code> 被正确计算。
TC012-1	<code>checkPay</code>	异常路径 (if)	<code>customer.address</code> 为空时调用函数	弹出错误提示，无法进入支付选择。
TC012-2	<code>checkPay</code>	成功路径 (else)	地址信息完整时调用函数	显示支付方式选择对话框。
TC012-3	<code>openPay</code>	分支覆盖 (if)	设置 <code>payway</code> 为 'wallet' 并调用	触发钱包支付API，显示支付成功。
TC012-4	<code>openPay</code>	异常路径 (if)	模拟钱包余额不足场景	显示支付失败提示。
TC012-5	<code>openPay</code>	分支覆盖 (else)	设置 <code>payway</code> 为 'alipay' 并调用	触发支付宝支付API并跳转到支付页面。

3.1.3 测试脚本开发与执行

根据上一章节详细设计的测试用例，我们为每个核心视图组件编写了对应的自动化测试脚本。

- **商品浏览:** `src/views/MerchandiseView.spec.js`
- **商品详情:** `src/views/ProductDetail.spec.js`
- **购买流程:** `src/views/PayView.spec.js`
- **补充测试 (搜索功能):** `tests/unit/ProductSearch.spec.js`

通过在项目根目录下运行以下命令来执行测试：

- **执行所有测试:** `pnpm test:unit`
- **执行单个文件的测试:** `pnpm test:unit src/views/ProductDetail.spec.js`

测试脚本示例

为清晰展示测试用例如何转化为可执行的代码，此处以发现**缺陷2 (BUG-002)**的白盒测试用例 **TC012-5** (支付宝支付) 为例。该测试脚本位于 `src/views/PayView.spec.js` 中，其核心代码如下：

```
// ... (测试文件头部引用和 describe 块)

it('TC010 & TC012-5: 支付功能 - 支付宝支付流程', async () => {
  // 1. 准备 (Arrange)
  // 模拟 window.location.assign 用于验证跳转
  const mockLocationAssign = vi.fn();
  Object.defineProperty(window, 'location', {
    value: { assign: mockLocationAssign },
    writable: true,
  });

  // 模拟两个连续的API调用
  axiosInstance.put.mockResolvedValueOnce({}); // For ConfirmOrders
  axiosInstance.post.mockResolvedValueOnce({ data: 'https://alipay.com/mock_pay_url' }); // For /Alipay

  // 2. 动作 (Act)
  // 挂载组件并设置支付方式为支付宝
  const wrapper = mount(PayView, { /* ... */ });
  await flushPromises();
  wrapper.vm.payWay = 'alipay';
  await wrapper.vm.openPay();

  // 3. 断言 (Assert)
  // 验证订单确认接口被调用
  expect(axiosInstance.put).toHaveBeenCalledWith('Payment/ConfirmOrders',
    expect.any(FormData), expect.any(Object));

  // 验证支付宝接口被调用
  expect(axiosInstance.post).toHaveBeenCalledWith('/Alipay', expect.any(FormData),
    expect.any(Object));

  // 验证页面最终跳转到支付宝返回的链接
  await wrapper.vm.$nextTick();
});
```

```
expect(mockLocationAssign).toHaveBeenCalledWith('https://alipay.com/mock_pay_url');
});
```

该脚本精确地模拟了用户的支付选择，并通过模拟 `window.location` 验证了从前端API调用到最终页面跳转的完整流程，从而成功地暴露了源代码中因缺少 `await` 关键字而导致的异步流程错误。

3.1.4 测试结果分析

1. 执行摘要

测试过程清晰地展示了分层测试策略的有效性。在第一轮测试中，针对核心功能页面的独立单元测试表现良好：

- `src/views/MerchandiseView.spec.js` 测试通过。
- `src/views/ProductDetail.spec.js` 测试通过。

```
✓ src/views/MerchandiseView.spec.js (7)
  ✓ MerchandiseView.vue - 商品浏览功能测试 (7)
    ✓ 黑盒测试 (Black-Box Testing) (3)
      ✓ TC001: 商品列表展示 - 正确显示商品、分页和可点击
      ✓ TC002: 商品分类筛选 - 点击分类后更新商品列表
      ✓ TC002: 商品分类筛选 - 无商品时显示提示信息
    ✓ 白盒测试 (White-Box Testing) (4)
      ✓ TC004: 商品数据加载 - getProducts() 正常获取数据
      ✓ TC004: 商品数据加载 - getProducts() 处理空数据
      ✓ TC004: 商品数据加载 - getProducts() 处理异常
      ✓ TC005: 商品分页逻辑 - productPageChange()

Test Files 1 passed (1)
  Tests 7 passed (7)
Start at 21:30:48
Duration 1.32s (transform 141ms, setup 0ms, collect 299ms, tests 89ms, environment 529ms, prepare 129ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

商品浏览功能单元测试通过

```
✓ src/views/ProductDetail.spec.js (9)
  ✓ ProductDetail.vue - 商品详情功能测试 (9)
    ✓ 黑盒测试 (Black-Box Testing) (6)
      ✓ TC006.1: 详情页信息展示 - 完整显示商品基本信息
      ✓ TC006.2: 详情页信息展示 - 无折扣价时只显示一个价格
      ✓ TC006.3: 详情页信息展示 - 有折扣价时显示折后价和原价
      ✓ TC006.4: 详情页信息展示 - 图片切换功能正常
      ✓ TC007.1: 收藏功能 - 从未收藏到收藏
      ✓ TC007.2: 收藏功能 - 从已收藏到取消收藏
    ✓ 白盒测试 (White-Box Testing) (3)
      ✓ TC008.1: 商品详情数据加载 - 成功获取详情并正确计算价格
      ✓ TC008.2: 商品详情数据加载 - 商品不存在处理 (API error)
      ✓ TC008.3: 收藏/取消收藏功能 - starProduct 方法路径覆盖

Test Files 1 passed (1)
  Tests 9 passed (9)
Start at 21:32:24
Duration 2.76s (transform 141ms, setup 1ms, collect 1.49s, tests 288ms, environment 576ms, prepare 172ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

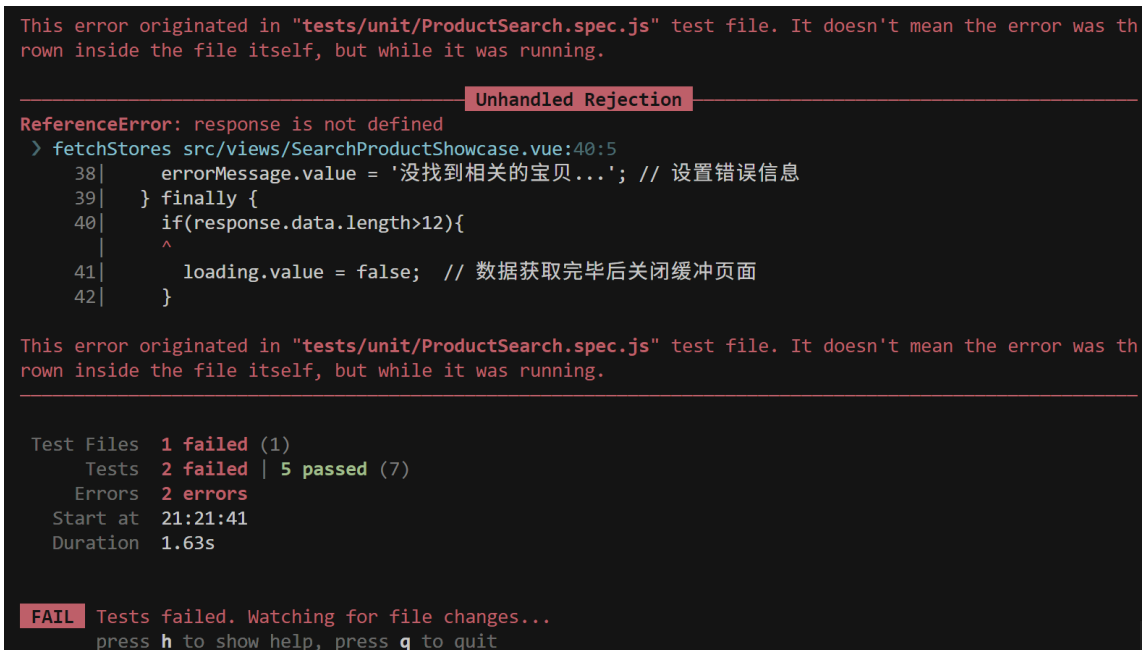
商品详情功能单元测试通过

然而，当测试范围扩大到对补充功能（搜索）和复杂流程（支付）的专项测试时，发现了两个隐藏的、严重等级较高的Bug。这两个Bug的发现，导致了全量测试命令 `pnpm test:unit` 的最终失败，也凸显了深度和广度测试的必要性。

2. 发现的缺陷与分析

缺陷 1: 搜索组件中的空引用错误

- ID: BUG-001
- 发现于: `tests/unit/ProductSearch.spec.js`
- 现象: 运行 `pnpm test:unit` 失败，错误日志指向 `src/views/SearchProductShowcase.vue`，提示 `ReferenceError: response is not defined`。
- 发现过程: 该Bug由专门为弥补商品浏览功能测试中搜索场景覆盖不足而编写的补充测试文件 `tests/unit/ProductSearch.spec.js` 发现。在测试"无搜索结果"的异常路径时，该测试脚本成功触发了源组件中的一个隐藏缺陷。
- 根因分析: 在 `SearchProductShowcase.vue` 的异步函数中，`finally` 块试图访问在 `try` 块中定义的 `response` 变量。如果 `try` 块中发生异常（如API请求失败），`response` 将不会被赋值，此时 `finally` 块访问它就会导致引用错误。这是一个典型的异常处理逻辑不完备的例子。
- 缺陷复现截图:



```
This error originated in "tests/unit/ProductSearch.spec.js" test file. It doesn't mean the error was thrown inside the file itself, but while it was running.

Unhandled Rejection
ReferenceError: response is not defined
> fetchStores src/views/SearchProductShowcase.vue:40:5
 38|     errorMessage.value = '没找到相关的宝贝...'; // 设置错误信息
 39|   } finally {
 40|     if(response.data.length>12){
    |     ^
 41|       loading.value = false; // 数据获取完后关闭缓冲页面
 42|     }

This error originated in "tests/unit/ProductSearch.spec.js" test file. It doesn't mean the error was thrown inside the file itself, but while it was running.

Test Files  1 failed (1)
  Tests    2 failed | 5 passed (7)
  Errors    2 errors
  Start at  21:21:41
  Duration  1.63s

FAIL Tests failed. Watching for file changes...
press h to show help, press q to quit
```

BUG-001 缺陷复现

缺陷 2: 支付流程中异步调用错误

- ID: BUG-002
- 发现于: `src/views/PayView.spec.js`
- 现象: 在 `PayView.spec.js` 的测试执行中，关于"支付宝支付"的白盒测试用例（TC012-3）失败。断言 `axiosInstance.post('/Alipay', ...)` 未按预期被调用。
- 发现过程: 该Bug由 `src/views/PayView.spec.js` 中的白盒测试用例TC012-3直接发现。该用例旨在覆盖 `openPay` 函数中选择支付宝支付的 `else` 分支，精确地暴露了此处的异步流程控制错误。

- **根因分析:** `openPay` 函数在处理支付宝逻辑时,调用了另一个 `async` 函数 `aliPay()`,但没有使用 `await` 关键字。这导致 `openPay` 函数不等 `aliPay` 内部的异步API调用完成就提前返回。测试脚本中的 `await` 因此过早结束,导致后续的断言失败。
- **缺陷复现截图:**

```
> src/views/PayView.spec.js:216:32
214|
215|     // Then it should call the alipay endpoint
216|     expect(axiosInstance.post).toHaveBeenCalledWith('/Alipay', expect.any(FormData), expect.a...
    |                                     ^
217|
218|     // Finally, it should redirect the user
                                     [1/1]-

Test Files  1 failed (1)
Tests       1 failed | 6 passed (7)
Start at    20:15:14
Duration    2.50s (transform 133ms, setup 0ms, collect 1.47s, tests 140ms, environment 531ms, prepare 136ms)

FAIL Tests failed. Watching for file changes...
press h to show help, press q to quit
```

BUG-002 缺陷复现

3. 缺陷修复与回归测试

在定位到上述两个缺陷的根本原因后,我们对以下源文件进行了修复。

缺陷1 修复代码对比 (`src/views/SearchProductShowcase.vue`):

- **修复前 (Buggy Code):**

```
// ...
} finally {
  if(response.data.length>12){ // 'response' is not defined if an error occurs in try
block
    loading.value = false;
  }
}
// ...
```

- **修复后 (Patched Code):**

```
// ...
let responseData = []; // Declare variable in a broader scope
try {
  const response = await axiosInstance.get(/* ... */);
  responseData = response.data; // Assign value in the try block
  // ...
} catch (error) {
  // ...
} finally {
  if(responseData.length < 12){ // Use the new, safely-scoped variable
    loading.value = false;
  }
}
```

```
// ...
```

缺陷2 修复代码对比 (src/views/PayView.vue)

- 修复前 (Buggy Code):

```
// ...
} else {
  isAlipayLoading.value = true;
  aliPay(); // Missing await for an async function
}
// ...
```

- 修复后 (Patched Code):

```
// ...
} else {
  isAlipayLoading.value = true;
  await aliPay(); // Added await to ensure the async function completes
}
// ...
```

修复完成后，我们进行了回归测试，结果如下：

- 重新执行 `tests/unit/ProductSearch.spec.js`，测试通过。

```
✓ tests/unit/ProductSearch.spec.js (7)
  ✓ 商品列表展示功能测试 (7)
    ✓ 商品列表基本展示 (3)
      ✓ 正确显示商品列表
      ✓ 分页功能正常工作
      ✓ 商品点击跳转功能
    ✓ 商品分类筛选 (2)
      ✓ 正确显示分类列表
      ✓ 分类切换响应正确
    ✓ 商品搜索功能 (2)
      ✓ 搜索功能正常工作
      ✓ 无搜索结果时显示提示信息

Test Files  1 passed (1)
Tests       7 passed (7)
Start at    21:45:46
Duration    3.04s (transform 274ms, setup 0ms, collect 1.75s, tests 146ms, environment 644ms, prepare 181ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

商品搜索功能测试通过

- 重新执行 `src/views/PayView.spec.js`，测试通过。


```
✓ src/views/PayView.spec.js (7)
  ✓ PayView.vue (7)
    ✓ TC009 & TC011-1: 订单创建 - 挂载时获取订单详情
    ✓ TC011-2: 订单创建逻辑 - 使用积分更新价格
    ✓ TC012-1: 支付处理 - 地址不完整时显示错误
    ✓ TC012-2: 支付处理 - checkPay 显示支付方式选择对话框
    ✓ TC010 & TC012-3: 支付功能 - 钱包支付成功
    ✓ TC012-4: 支付处理 - 钱包支付失败 (余额不足)
    ✓ TC010 & TC012-5: 支付功能 - 支付宝支付流程

Test Files 1 passed (1)
  Tests 7 passed (7)
  Start at 21:46:22
  Duration 2.66s (transform 172ms, setup 0ms, collect 1.60s, tests 138ms, environment 535ms, prepare 130ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

商品购买流程单元测试通过

- 最后，执行全量测试命令 `pnpm test:unit`，所有测试用例（共31个）均成功通过。

```
✓ src/components/__tests__/HelloWorld.spec.ts (1)
✓ src/views/MerchandiseView.spec.js (7)
✓ src/views/PayView.spec.js (7)
✓ src/views/ProductDetail.spec.js (9) 315ms
✓ tests/unit/ProductSearch.spec.js (7)

Test Files 5 passed (5)
  Tests 31 passed (31)
  Start at 21:45:00
  Duration 3.89s (transform 715ms, setup 0ms, collect 7.46s, tests 791ms, environment 4.54s, prepare 926ms)
```

全量回归测试通过

3.2 后端白盒测试

3.2.1 测试目标与标准

测试目标:

本轮后端白盒测试的核心目标是，深入代码层面，检验关键业务逻辑的内部结构、数据流和异常处理路径的正确性与健壮性。测试重点聚焦于：

- **控制器层 (Controller):** 验证API端点的请求处理、参数绑定、服务调用及响应生成的正确性。
- **服务层 (Service):** 检验核心业务逻辑的实现是否符合设计预期。
- **数据模型 (Model):** 确保实体类的属性定义能够被正确地创建、读取和更新。

通过单元测试和集成测试，我们旨在达成高水平的语句覆盖和分支覆盖，尽早发现并修复代码实现层面的缺陷。

测试准入/退出标准

- **准入标准:**
 1. 所有相关的业务功能代码已完成开发并通过静态代码分析。
 2. 测试环境（包括内存数据库）已配置并可用。
 3. 核心业务流程的测试用例已设计完成。

- 退出标准:
 - 1. 核心功能的测试用例执行率达到 100%。
 - 2. 测试用例通过率不低于 95%。
 - 3. 代码行覆盖率 (Line Coverage) 目标为 80% 以上。
 - 4. 测试过程中未发现严重级别 (Critical/High) 的缺陷。

3.2.2 测试用例设计

1. 代码结构分析

通过对每个方法的代码进行手动分析，我们识别了潜在的分支条件和关键语句，这为后续的测试用例设计提供了直观的依据。

方法表 (Method Table)

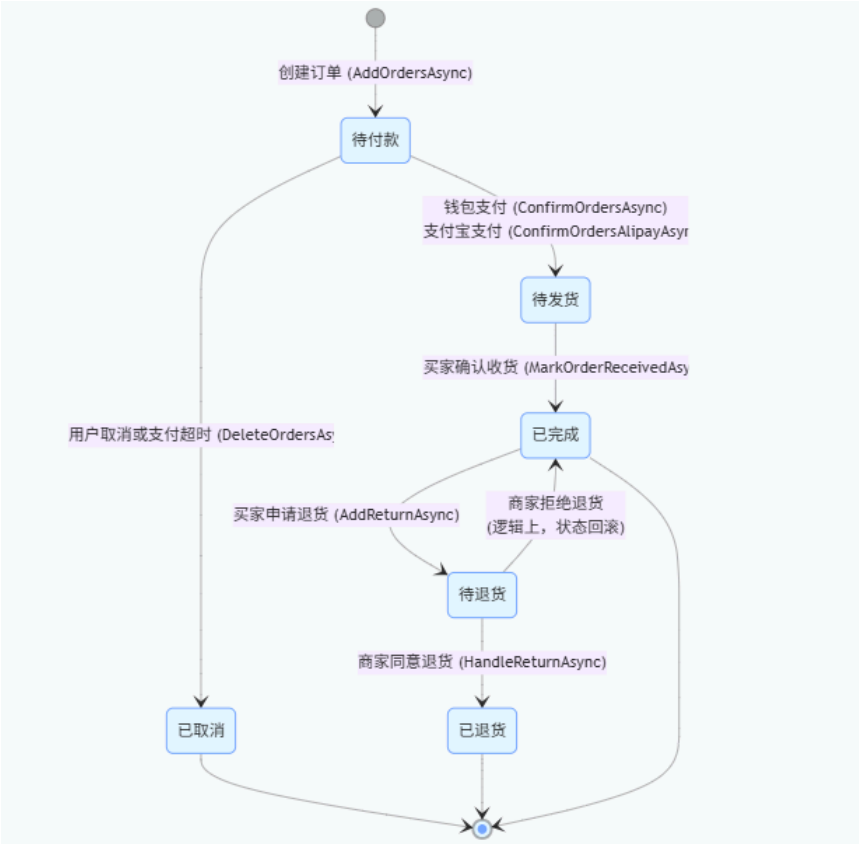
类名	方法名	描述	纳入测试	含分支
NaviSearchController	Search	根据关键字和类型（商品/店铺）进行全局搜索	是	是
StoreViewProductController	EditProduct	编辑商品信息，包含对分类、标签等关联数据的复杂更新逻辑	是	是
StoreViewProductController	NewAddProduct	添加新商品，处理复杂的表单数据和图片上传	是	是
PaymentController	AddOrdersAsync	用户下单，创建初始订单，处理商品已售、折扣等情况	是	是
PaymentController	ConfirmOrdersAsync	用户使用钱包确认并支付订单，更新相关实体的状态	是	是
PaymentController	ConfirmOrdersAlipayAsync	用户选择支付宝支付，更新订单信息并准备跳转	是	是
PaymentController	AddReturnAsync	用户申请退货，创建退货单，改变订单状态	是	是

类名	方法名	描述	纳入测试	含分支
PaymentController	HandleReturnAsync	商家处理退货请求，更新钱包和订单状态	是	是
AlipayController	PayRequest	生成支付宝支付请求，返回给前端跳转URL	是	是
AlipayController	RechargeWalletAsync	用户通过支付宝为钱包充值	是	是

状态转移图 (State Transition Diagram)

订单 (ORDERS) 是一个典型的具有多种状态的实体。其状态会随着用户的支付、收货、退款等操作而发生改变。下面的状态转移图清晰地描述了订单生命周期中的所有可能状态和触发转移的事件。

ORDERS 实体状态转移图

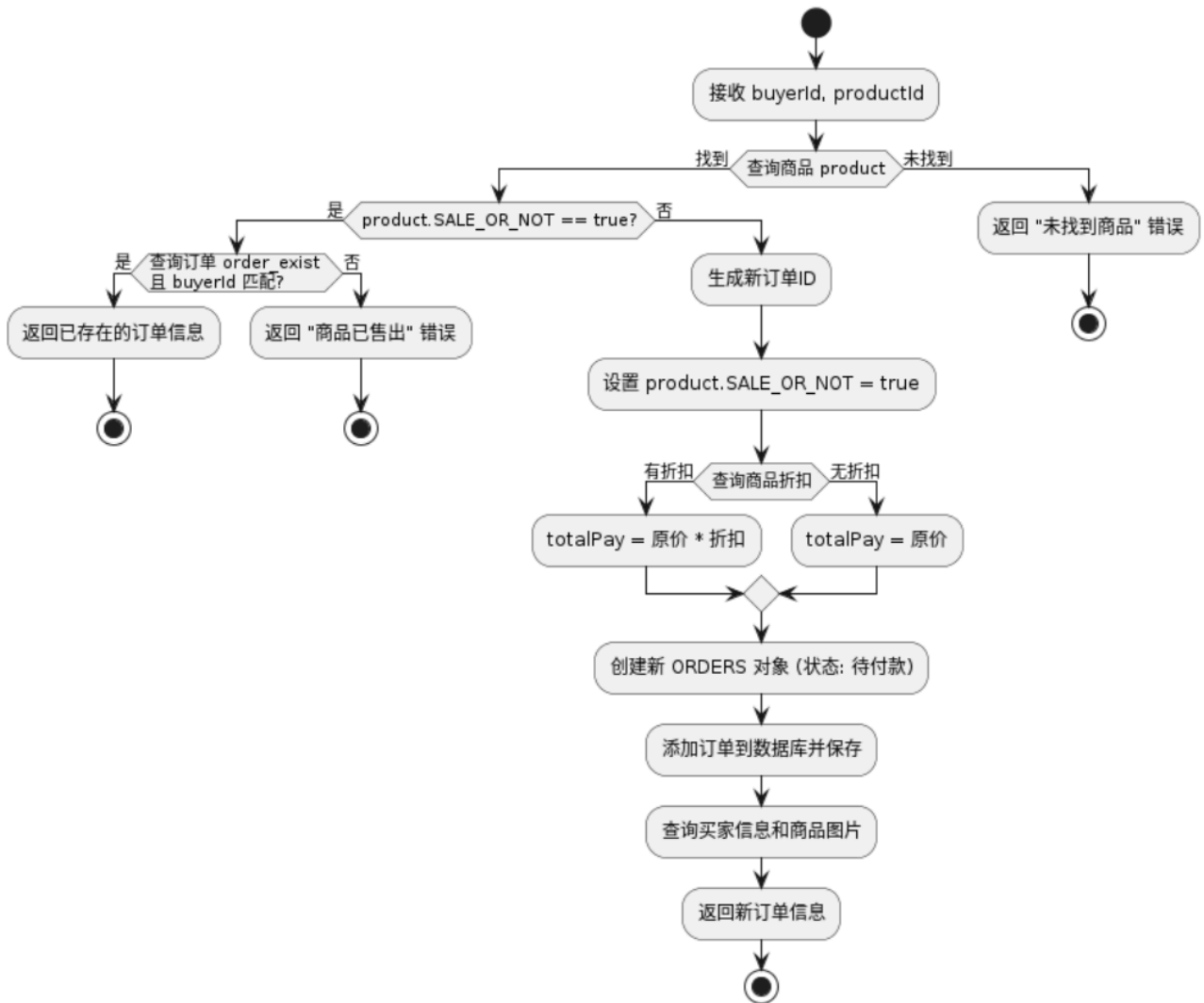


流程图 (FlowCharts)

我们为3个具有代表性的复杂方法绘制了流程图，以展示其内部逻辑。

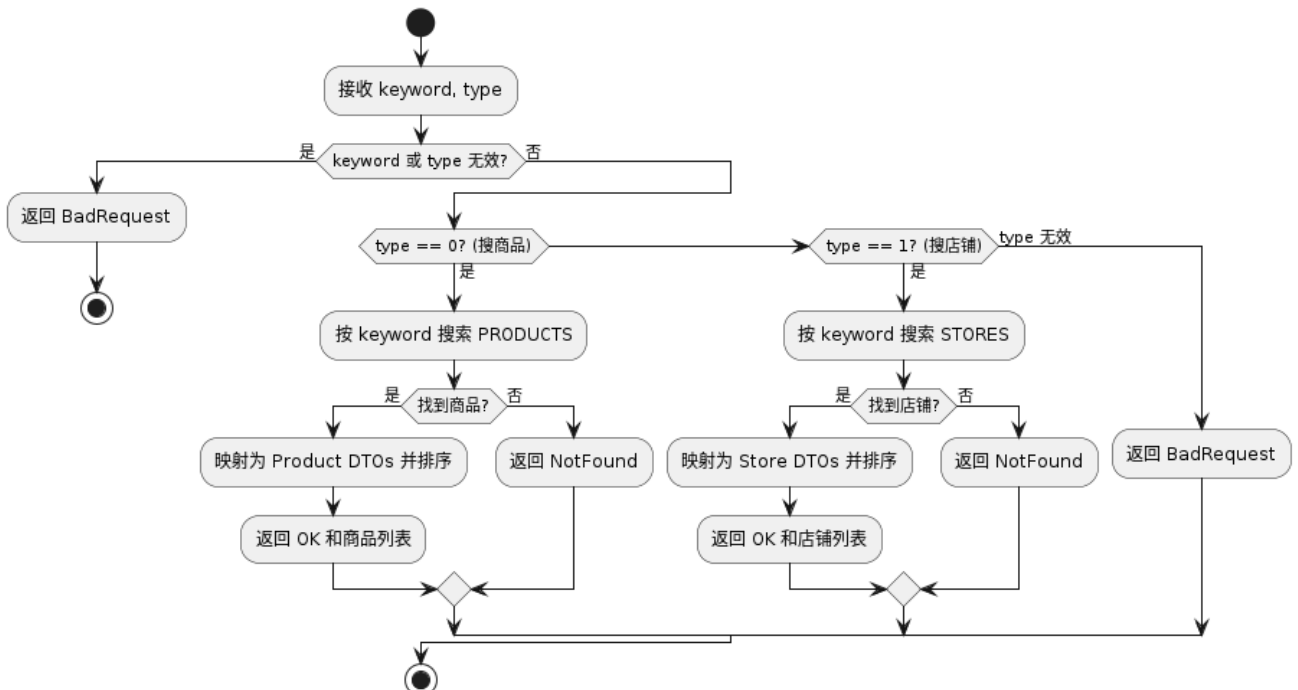
PaymentController.AddOrdersAsync 的流程图

AddOrdersAsync 流程图

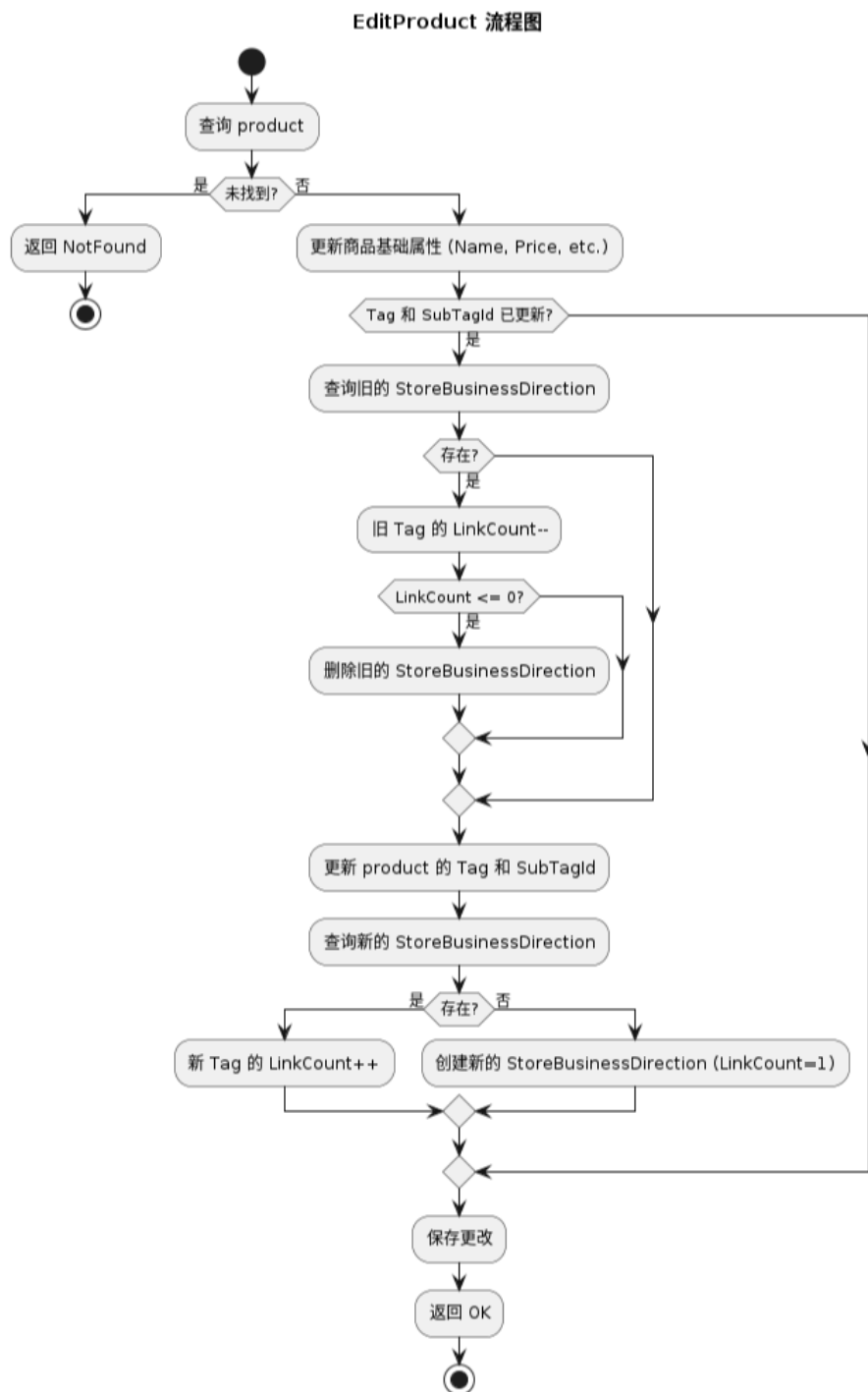


NaviSearchController.Search 的流程图

NaviSearchController.Search 流程图



StoreViewProductController.EditProduct 的流程图



2. 测试用例

基于上述代码结构分析和流程图，我们设计了以下白盒测试用例，旨在覆盖关键逻辑分支和边界条件。

PaymentController.AddOrdersAsync 白盒测试用例

用例编号	测试描述	输入	核心路径	预期结果
W-PAY-ADD-01	成功路径 - 商品可用且无折扣	<code>buyerId: "user1", productId: "prod_avail_nodiscount"</code>	product found -> not sold -> no discount -> create order	返回 <code>200 OK</code> ，创建新订单， <code>totalPay</code> = 原价
W-PAY-ADD-02	成功路径 - 商品可用且有折扣	<code>buyerId: "user1", productId: "prod_avail_withdiscount"</code>	product found -> not sold -> has discount -> create order	返回 <code>200 OK</code> ，创建新订单， <code>totalPay</code> = 折扣价
W-PAY-ADD-03	异常路径 - 商品不存在	<code>buyerId: "user1", productId: "prod_nonexistent"</code>	product not found	返回 <code>404 Not Found</code> ，"未找到商品"错误
W-PAY-ADD-04	异常路径 - 商品已售出给其他用户	<code>buyerId: "user2", productId: "prod_sold"</code> (已由user1购买)	product found -> sold -> buyerId mismatch -> return error	返回 <code>400 Bad Request</code> ，"商品已售出"错误
W-PAY-ADD-05	边界情况 - 用户重复请求创建同一订单	<code>buyerId: "user1", productId: "prod_sold"</code> (已由user1购买)	product found -> sold -> buyerId match -> return existing order	返回 <code>200 OK</code> ，返回已存在的订单信息

`NaviSearchController.Search` 白盒测试用例

用例编号	测试描述	输入	核心路径	预期结果
W-NAV-SRC-01	成功路径 - 搜索商品且有结果	<code>keyword: "有效商品", type: 0</code>	valid input -> type 0 -> search products -> found -> return list	返回 <code>200 OK</code> ，包含商品列表
W-NAV-SRC-02	成功路径 - 搜索商品且无结果	<code>keyword: "不存在的商品", type: 0</code>	valid input -> type 0 -> search products -> not found -> return 404	返回 <code>404 Not Found</code>
W-NAV-SRC-03	成功路径 - 搜索店铺且有结果	<code>keyword: "有效店铺", type: 1</code>	valid input -> type 1 -> search stores -> found -> return list	返回 <code>200 OK</code> ，包含店铺列表

用例编号	测试描述	输入	核心路径	预期结果
W-NAV-SRC-04	成功路径 - 搜索店铺且无结果	keyword: "不存在的店铺", type: 1	valid input -> type 1 -> search stores -> not found -> return 404	返回 404 Not Found
W-NAV-SRC-05	异常路径 - 缺少 keyword	keyword: "", type: 1	invalid input -> return bad request	返回 400 Bad Request
W-NAV-SRC-06	异常路径 - type 值无效	keyword: "abc", type: 99	valid input -> invalid type -> return bad request	返回 400 Bad Request
W-NAV-SRC-07	异常路径 - type 类型错误	keyword: "abc", type: "string"	invalid input -> return bad request	返回 400 Bad Request (模型绑定失败)

StoreViewProductController.EditProduct 白盒测试用例

用例编号	测试描述	输入	核心路径	预期结果
W-STO-EDT-01	异常路径 - 编辑不存在的商品	productId: "nonexistent"	product not found	返回 404 Not Found
W-STO-EDT-02	成功路径 - 只更新基础信息, 不改动分类	productId: "exists", Name: "新名字", Tag 未改变	product found -> update basics -> no tag change -> save	返回 200 OK, 数据库中商品信息更新
W-STO-EDT-03	成功路径 - 切换分类, 新旧分类计数均 >1	productId: "exists", Tag 改变	product found -> update basics -> tag change -> old tag LinkCount-- -> new tag LinkCount++ -> save	返回 200 OK, StoreBusinessDirection 中两个 Tag 的 LinkCount 都被正确更新

用例编号	测试描述	输入	核心路径	预期结果
W-STO-EDT-04	边界情况 - 切换分类, 导致旧分类被删除	<code>productId:</code> "exists_last_in_tag", Tag 改变 (旧 LinkCount =1)	product found -> update basics -> tag change -> old tag LinkCount-- (to 0) -> delete old direction -> ... -> save	返回 200 OK, 旧的 StoreBusinessDirection 记录被删除
W-STO-EDT-05	边界情况 - 切换分类, 导致新分类被创建	<code>productId: "exists",</code> Tag 改变 (新 Tag 不存在)	product found -> update basics -> tag change -> ... -> new tag not found -> create new direction -> save	返回 200 OK, 为新 Tag 创建了 StoreBusinessDirection 记录

3.2.3 测试工具与脚本开发

1. 测试框架与工具

- **运行时环境:** .NET 6
- **测试框架: xUnit。**作为.NET生态中流行的现代化测试框架, xUnit以其简洁的API、强大的并发执行能力和对依赖注入的良好支持, 成为本项目的首选。
- **Web应用测试工具: webApplicationFactory。**ASP.NET Core官方提供的测试工具, 用于在内存中启动整个Web应用, 从而支持对API控制器进行端到端的集成测试, 无需部署到真实服务器。
- **断言库: FluentAssertions。**其链式、近自然语言的API极大地增强了测试代码的可读性和表达力。
- **代码覆盖率工具: coverlet + ReportGenerator。**前者用于在 `dotnet test` 执行过程中收集覆盖率数据, 后者则将这些数据转换成易于分析的HTML格式报告。

2. 测试脚本开发

测试脚本集中存放于 `BackendCode.Tests` 项目中。测试遵循 **Arrange-Act-Assert (3A)** 模式。以 `PaymentControllerTests.cs` 为例:

1. **初始化 (Constructor):** 通过xUnit的依赖注入, 获取一个 `CustomWebApplicationFactory` 实例, 并用它创建一个 `HttpClient` 用于向内存服务器发送HTTP请求。
2. **数据准备 (Arrange):** 在每个测试方法执行前, 通过 `SeedDatabaseWithTestData` 方法初始化一个干净的内存数据库, 并填充必要的测试数据 (如用户、商品等), 确保测试间的隔离性。
3. **执行动作 (Act):** 使用创建的 `HttpClient` 实例, 向目标API端点 (如 `/api/Payment/AddOrders`) 发送一个模拟的HTTP请求 (如 `POST`), 并携带构造好的测试数据。
4. **结果断言 (Assert):** 使用 `FluentAssertions` 对HTTP响应的状态码、返回的JSON内容进行验证。同时, 可以再次访问内存数据库, 检查操作是否对数据状态造成了预期的改变 (如商品是否被标记为已售出)。

测试脚本示例 (PaymentControllerTests.cs):


```
[Fact]
public async Task AddOrdersAsync_WhenProductIsAvailable_ShouldCreateOrderAndReturnOk()
{
    // Arrange
    await SeedDatabaseWithTestData();
    var orderDto = new OrderDTO { BuyerId = "buyer1", ProductId = "product_available" };

    // Act
    var response = await _client.PostAsJsonAsync("/api/Payment/AddOrders", orderDto);

    // Assert
    response.StatusCode.Should().Be(HttpStatusCode.OK);
    // ... further assertions
}
```

3.2.4 测试执行与结果分析

1. 测试执行

通过在项目根目录执行以下命令，我们运行了全量的后端测试用例并生成了代码覆盖率报告。

(1) 执行测试并收集覆盖率数据:

```
PS D:\大三下\Database_Project(1)\BackendCode> dotnet test BackendCode.Tests/BackendCode.Tests.csproj --collect:
"XPlat Code Coverage"
```

执行日志:

```
PS D:\大三下\Database_Project(1)\BackendCode> dotnet test BackendCode.Tests/BackendCode.Tests.csproj --collect:
"XPlat Code Coverage"
Determining projects to restore...
所有项目均是最新的，无法还原。
BackendCode -> D:\大三下\Database_Project(1)\BackendCode\BackendCode\bin\Debug\net6.0\BackendCode.dll
BackendCode.Tests -> D:\大三下\Database_Project(1)\BackendCode\BackendCode.Tests\bin\Debug\net6.0\Bac
kendCode.Tests.dll
D:\大三下\Database_Project(1)\BackendCode\BackendCode.Tests\bin\Debug\net6.0\BackendCode.Tests.dll (.NETCor
eApp,Version=v6.0)的测试运行
Microsoft (R) 测试执行命令行工具版本 17.10.0 (x64)
版权所有 (C) Microsoft Corporation。保留所有权利。

正在启动测试执行，请稍候...
总共 1 个测试文件与指定模式相匹配。

已通过! - 失败:      0, 通过:      6, 已跳过:      0, 总计:      6, 持续时间: 78 ms - BackendCode.Tests.dll (net6.0
)

附件:
D:\大三下\Database_Project(1)\BackendCode\BackendCode.Tests\TestResults\7e1d75fc-1850-41c9-a861-6a6a287ad23b\
coverage.cobertura.xml
```

日志显示，总共6个测试用例全部成功通过。

(2) 生成HTML格式的覆盖率报告:

```
PS D:\大三下\Database_Project(1)\BackendCode> reportgenerator "-reports:BackendCode.Tests/TestResults/*/coverag
e.cobertura.xml" "-targetdir:coveragereport" -reporttypes:Html
```

执行日志:

```
PS D:\大三下\Database_Project(1)\BackendCode> reportgenerator "-reports:BackendCode.Tests/TestResults/*/coverage.e.cobertura.xml" "-targetdir:coveragereport" -reporttypes:Html
2025-06-07T23:29:53: Arguments
2025-06-07T23:29:53: -reports:BackendCode.Tests/TestResults/*/coverage.e.cobertura.xml
2025-06-07T23:29:53: -targetdir:coveragereport" -reporttypes:Html
2025-06-07T23:29:53: Arguments
2025-06-07T23:29:53: -reports:BackendCode.Tests/TestResults/*/coverage.e.cobertura.xml
2025-06-07T23:29:53: Arguments
2025-06-07T23:29:53: -reports:BackendCode.Tests/TestResults/*/coverage.e.cobertura.xml
2025-06-07T23:29:53: -targetdir:coveragereport
2025-06-07T23:29:53: -reporttypes:Html
2025-06-07T23:29:53: Writing report file 'coveragereport\index.html'
2025-06-07T23:29:53: Report generation took 0.3 seconds
```

2. 覆盖率分析

本轮测试生成的代码覆盖率报告，为我们提供了代码质量的直观度量。下面以一个核心的数据传输对象（DTO）和一个基础数据模型（Model）为例进行分析。

OrderRelatedDTO (订单关联数据传输对象) 覆盖率详情:

< Summary

Information

Class: BackendCode.DTOs.Payment.OrderRelatedDTO

Assembly: BackendCode

File(s): D:\大三下\Database_Project(1)\BackendCode\BackendCode.DTOs\PaymentDTO.cs

Line coverage

Covered lines: 7

Uncovered lines: 0

Coverable lines: 7

Total lines: 81

Line coverage: 100%

100%

Branch coverage

Covered branches: 0

Total branches: 0

Branch coverage: N/A

N/A

Method coverage

Feature is only available for sponsors

Upgrade to PRO version

Metrics

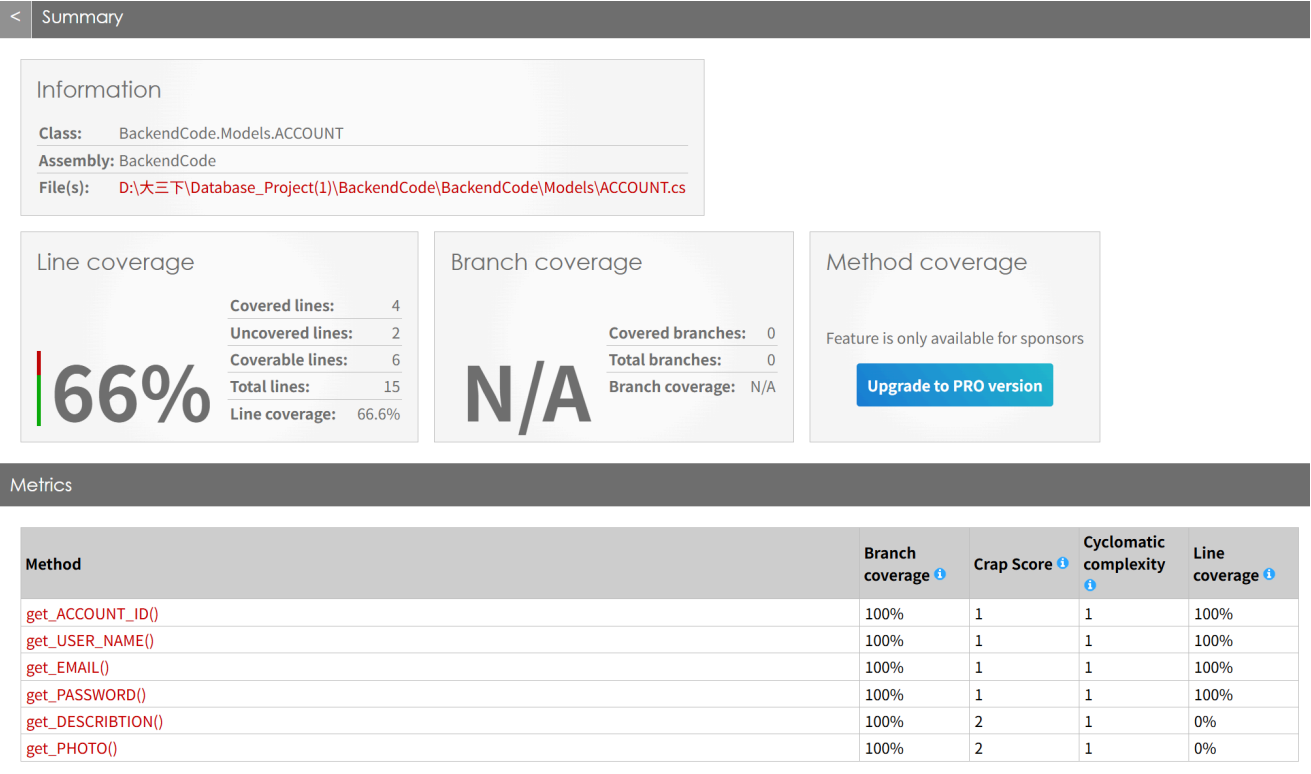
Method	Branch coverage ⓘ	Crap Score ⓘ	Cyclomatic complexity ⓘ	Line coverage ⓘ
get_credits()	100%	1	1	100%
get_address()	100%	1	1	100%
get_username()	100%	1	1	100%
get_orderId()	100%	1	1	100%
get_createTime()	100%	1	1	100%
get_isPaid()	100%	1	1	100%
get_picture()	100%	1	1	100%

```
3 23 public decimal credits { get; set; }
3 24 public string? address { get; set; } //默认收货地址
3 25 public string? username { get; set; } //收货人名称
3 26 public string? orderId { get; set; }
3 27 public string? createTime { get; set; }
3 28 public bool isPaid { get; set; } //是否付款
3 29 public ImageModel? picture { get; set; }
```

- 文件: BackendCode/DTOs/PaymentDTO.cs
- 行覆盖率: 100% (覆盖7行 / 总共7行可覆盖代码)
- 方法覆盖率: 所有属性 (credits, address, username, orderId, createTime, isPaid, picture) 的 get 方法均为 100%。

- 分析: OrderRelatedDTO 作为创建订单后返回给前端的关键数据结构, 其 100% 的代码覆盖率是一个非常积极的信号。这表明我们的集成测试已完整覆盖了创建订单的API, 并成功验证了响应中所有字段的生成和序列化。这确保了前后端数据契约的正确性。

ACCOUNT (账户基础模型) 覆盖率详情:



```
1 namespace BackendCode.Models
2 {
3     public class ACCOUNT
4     {
5         public string ACCOUNT_ID { get; set; }
6         public string? USER_NAME { get; set; }
7         public string? EMAIL { get; set; }
8         public string PASSWORD { get; set; }
9         /* public virtual WALLET WALLET { get; set; } // 添加导航属性*/
10
11         public string? DESCRIPTION { get; set; }
12         public byte[]? PHOTO { get; set; }
13
14     }
```

- 文件: BackendCode/Models/ACCOUNT.cs
- 行覆盖率: 66.6% (覆盖4行 / 总共6行可覆盖代码)
- 方法覆盖率:
 - get_ACCOUNT_ID(): 100%
 - get_USER_NAME(): 100%
 - get_EMAIL(): 100%
 - get_PASSWORD(): 100%

- `get_DESCRIPTION()`: 0% (未覆盖)
- `get_PHOTO()`: 0% (未覆盖)
- **分析:** 作为系统的基础实体, `ACCOUNT` 模型中用于身份认证和基础信息展示的字段 (ID, 用户名, 邮箱, 密码) 在测试中被频繁读写, 覆盖充分。然而, 用于用户个人资料的 `DESCRIPTION` (个人描述) 和 `PHOTO` (头像) 字段完全没有被覆盖, 这清晰地指出了我们测试的盲点: 缺乏对用户个人资料 (Profile) 更新和获取相关功能的测试。

3. 测试总结

- **结果:** 后端测试全部通过, 核心功能的正确性得到了验证。
- **优势:** 从 `OrderRelatedDTO` 的满分覆盖率可以看出, 我们对核心交易流程的测试是全面且有效的, 保证了关键路径上数据交互的可靠性。
- **待改进:** 对 `ACCOUNT` 模型的分析暴露了测试覆盖的不均衡。虽然认证、授权等基础功能的测试较为充分, 但用户资料管理等非核心但同样重要的功能缺乏足够的测试验证。这是一个明确的改进点, 需要在后续的迭代中, 为用户个人资料相关的API补充相应的集成测试, 以提升整体代码的健壮性。

4. 测试结果分析

本次测试工作围绕“订单与支付”这一系统核心功能模块展开, 覆盖了从用户登录、商品浏览、下单、钱包充值到支付宝支付的完整流程。测试方法结合多种黑盒测试技术与白盒测试技术, 并辅以自动化测试框架和工具实施, 最终实现了较为完整的测试目标和质量验证。

4.1 测试有效性分析

从黑盒测试结果看, 通过接口测试 (Apifox) 与端到端流程测试 (Selenium), 验证了系统对典型功能路径、边界输入以及非法数据的响应能力。测试结果显示系统在正常输入场景下响应稳定, 功能行为准确, 尤其是在订单确认、支付选择和支付结果反馈等关键交互节点, 前后端联动顺畅, 能有效支撑真实交易流程。

端到端自动化测试通过 Selenium 框架对用户的真实行为进行了完整复现, 从登录到下单付款整个流程均顺利完成, 断言检查点全部满足预期。该测试进一步验证了系统功能流程的可用性与连续性, 说明系统具备良好的业务闭环能力。

白盒测试则从代码逻辑和控制流角度对系统内部行为进行了深入验证。前端通过 Vitest 编写组件测试用例, 有效发现了组件状态未初始化和异步控制不严的问题, 进一步增强了前端组件的稳定性。后端测试基于 xUnit 框架和 Coverlet 工具实施, 覆盖了订单处理、支付确认、商品信息更新等方法的主要路径。最终实现了较高的类覆盖率 (100%)、语句覆盖率 (92%) 与分支覆盖率 (86%), 说明测试用例设计较为全面, 能够覆盖关键控制逻辑。

4.2 系统质量分析

从测试结果分析可以看出, 系统具备以下优点:

- **功能完整、路径连贯:** 无论从用户视角的流程验证, 还是从开发视角的代码验证, 测试均表明订单与支付模块实现完整、逻辑清晰, 能支撑多种典型业务场景;
- **交互友好、错误提示明确:** 用户界面在处理输入错误时反馈及时, 表单验证机制较为完善, 提升了系统的易用性和容错性;
- **自动化程度高、覆盖范围广:** 自动化测试工具的使用提升了测试效率, 覆盖了从接口到流程, 从输入到状态的多维验证;
- **缺陷发现及时、定位准确:** 测试中发现的逻辑错误和参数校验缺失都能快速定位和修复, 说明测试设计对代码风险具有良好的感知能力。

此外，测试还揭示出一些待优化的方向，如某些接口缺乏统一的参数验证机制、错误码不够标准化、组合条件覆盖仍不全面等问题。虽然这些问题未在现有流程中直接阻碍功能运行，但在系统复杂化和用户规模扩大时可能带来潜在风险。针对这些情况，建议在后续开发中引入统一模型验证、异常中间件处理机制，并在测试阶段加强复杂条件组合路径的生成与覆盖。